

Social coding and open software - What can you do to get credit for your code and to allow reuse

In this lesson we will discuss how and why to share code and data, what kind of licenses are used in what situation, and how software can be cited.

We will try to connect software licenses to FAIR principles, give practical recommendations for starting, contributing, and reusing code and help you navigating and deciding on licenses.

📌 Why software licenses matter

- You **find some great code** or data that you want to reuse for your own publication (good for the original author: you will cite them and maybe other people who cite you will cite them).
- You need to **modify the code** a little bit, or you remix the data a bit.
- When it comes time to publish, you realize there is no **license**.

Now we have a problem:

- You manage to **publish the paper without the software/data** but others cannot build on your software and data and you don't get as many citations as you could.
- Or, you **cannot publish it at all** if the journal requires that papers should come with data and software so that they are reproducible.

This lesson is about how to avoid this situation for you and others.

⚙️ Prerequisites

No computational prerequisites required for this lesson.

20 min	Social coding
90 min	Software licensing focusing on open source
20 min	Software citation
10 min	Sharing data

Social coding

📌 Objectives

Question 1: Why would I want to share my scripts/code/data?

****Choose many****. Vote by adding an `o` character:

- A: Easier to find and reproduce (scientific reproducibility)
 - votes:
- B: More trustworthy: others can verify correctness and find and report bugs
 - votes:
- C: Enables others to build on top of your code (derivative work, provided the license allows it)
 - votes:
- D: Others can submit features/improvements
 - votes:
- E: Others can help fixing bugs
 - votes:
- F: Many tools and apps are free for open source, so no financial cost for this (GitHub, GitLab, Appveyor, Read the Docs)
 - votes:
- G: Good for your CV: you can show what you have built
 - votes:
- H: Discourages competitors. If others can't build on your work, they will make competing work
 - votes:
- I: When publicly shared, usually no time-stamp or set a version

Question 1: Why would I want to share my scripts/code/data?

****Choose many****. Vote by adding an `o` character:

- A: Easier to find and reproduce (scientific reproducibility)
- votes:
- B: More trustworthy: others can verify correctness and find and report bugs
- votes:
- C: Enables others to build on top of your code
(derivative work, provided the license allows it)
- votes:
- D: Others can submit features/improvements
- votes:
- E: Others can help fixing bugs
- votes:
- F: Many tools and apps are free for open source, so no financial cost for this
(GitHub, GitLab, Appveyor, Read the Docs)
- votes:
- G: Good for your CV: you can show what you have built
- votes:
- H: Discourages competitors. If others can't build on your work,
they will make competing work
- votes:
- I: When publicly shared, usually we time-stamp or set a version,
so it is easier to refer to a specific version
- votes:
- J: You can reuse your own code later after change of job or affiliation
- votes:
- K: It encourages me to code properly from the start
- votes:

Question 2: The most concerning thing for me, If I share my software now

****Choose one****. Vote by adding an `o` character:

- A: It will be scooped (stolen) by someone else
- votes:
- B: It will expose my "ugly code"
- votes:
- C: Others may find bugs and mistakes. What if the algorithm is wrong?
- votes:
- D: I will get too many questions, I do not have time for that
- votes:
- E: Losing control over the direction of the project
- votes:
- F: Low quality copies will appear
- votes:
- G: I won't be able to sell this later. Someone else will make money from it
- votes:
- H: It is too early, I am just prototyping, I will write version to distribute later
- votes:
- I: Worried about licensing and legal matters, as they are very complicated
- votes:

Question 3: Why is software often treated differently from papers?

Free-form answers:

- ...
- ...
- ...
- ...
- ...
- ...

Question 4: When you find a repository with code/library you would like to reuse, what are the things you look at to decide whether you use it?

Free-form answers:

- ...
- ...
- ...
- ...
- ...
- ...

```

- G: I won't be able to sell this later. Someone else will make money from it
  - votes:

- H: It is too early, I am just prototyping, I will write version to distribute later
  - votes:

- I: Worried about licensing and legal matters, as they are very complicated
  - votes:

```

Question 3: Why is software often treated differently from papers?

Free-form answers:

```

- ...
- ...
- ...
- ...
- ...
- ...

```

Question 4: When you find a repository with code/library you would like to reuse, what are the things you look at to decide whether you use it?

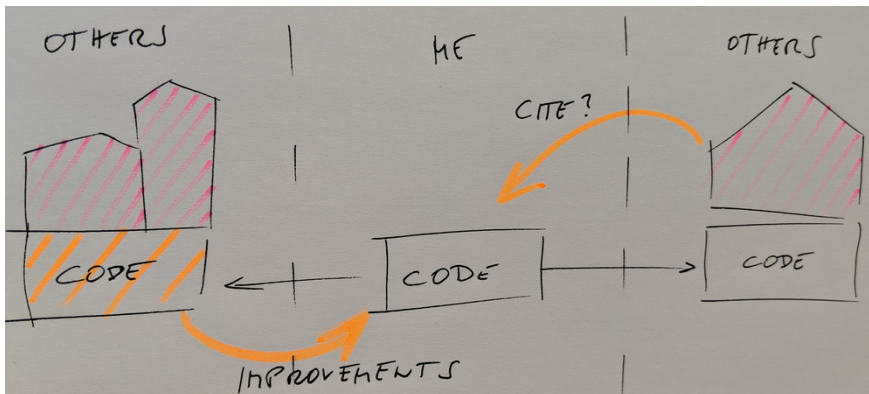
Free-form answers:

```

- ...
- ...
- ...
- ...
- ...
- ...

```

- The goal is maximum visibility and maximum reuse.
- The more interesting science is done referencing my paper, the better for me.
- Nobody actively tries to limit the reach of their papers.



Different ways we can benefit from sharing code.

Sharing code:

- “I did all the ground work and they get to do the interesting science?”
- Sharing code and encouraging *derivative work* may boost your academic impact.

However a study showed that despite these policies, many people still do not share their code 😞. This paper includes samples of charming author responses such as:

Journal policies as motivation for sharing

“When you approach a PI for the source codes and raw data, you better explain who you are, whom you work for, why you need the data and what you are going to do with it.”
From Science editorial policy

Motivation for open source software

“We require that all computer code used for modeling and/or data analysis that is not

Instructor note available be deposited in a publicly accessible repository upon publication.

In rare exceptional cases where security concerns or competing commercial interests pose a conflict, code-sharing arrangements that still facilitate reproduction of the work should be discussed with your Editor no later than the revision stage.”

- Enable derivative work

Do not lock yourself out of own code

- Attract developers who want to be able to show the coding work on their CVs
- Tighter regulated domains (e.g. medicine) researchers should be able to replicate and build upon the work of others (PSS) can lead to more engagement from industry. Research may lead to new products
- Journals that authors are required to make **materials, data, code, and associated protocols** promptly available to readers extend due qualifications. Any restrictions on the availability of materials or information must be disclosed to the editors at the time of

Sharing software is also scary so be disclosed in the submitted manuscript.”

Instructor note

How will your work be visible if it is used by others, many people?

However, a study showed that despite these policies, many people still do not share their code 😞. This paper includes samples of charming author responses such as:

Journal policies as motivation for sharing

"When you approach a PI for the source codes and raw data, you better explain who you are, whom you work for, why you need the data and what you are going to do with it."

Motivation for open source software

"We require that all computer code used for modeling and/or data analysis that is not

Instructor note available be deposited in a publicly accessible repository upon publication.

In rare exceptional cases where security concerns or competing commercial interests pose a conflict, code-sharing arrangements that still facilitate reproduction of the work should be discussed with your Editor no later than the revision stage."

- Enable derivative work

Do not lock yourself out of own code

- Attract developers who want to be able to show the coding work on their CVs
- Tight regulation of domain publication open source
- Open the code software (OSS) can lead to more engagement from industry, which may lead to more impact
- Journal authors are required to make materials, data, code, and associated protocols promptly available to become standard
- Indue qualifications. Any restrictions on the availability of materials or information must be disclosed to the editors at the time of

Sharing software is also scary

Instructor note

We revisit answers to Question 2 (above).

- **Fear of being scooped:** A license can avoid it, and you can release when you are ready. Anyway, it is very unlikely that others will understand your code and publish before you without involving you in a collaboration. Sharing is a form of publishing.
- **Exposes possibly "ugly code":** In practice almost nobody will judge the quality of your code. "Software, once written, is never really finished" (N. Asparouhova).
- **Others may find bugs and mistakes:** Isn't this good? Would you not like to use a code which gives people the chance to locate bugs? If you don't release, people will assume there are bugs anyway.
- **Others may require support and ask too many questions:** This can become a problem: use tools and community and protect your time. You aren't required to support anyone. You can also "archive" a repository to disable most forms of interaction (issues, PRs). Also a note in README on support level helps.
- **Fear of losing control over the direction of the project:** Open source does not mean everybody can change your version.
- **"Bad" derivative projects may appear:** It will be clear which is the official version.

Code reusability: What contributes to reusability?

What contributes to you being able to reuse stuff that others make, and others (or you) being able to reuse your stuff? When you find a repository with code you would like to reuse, you may look at the following things to determine its reusability:

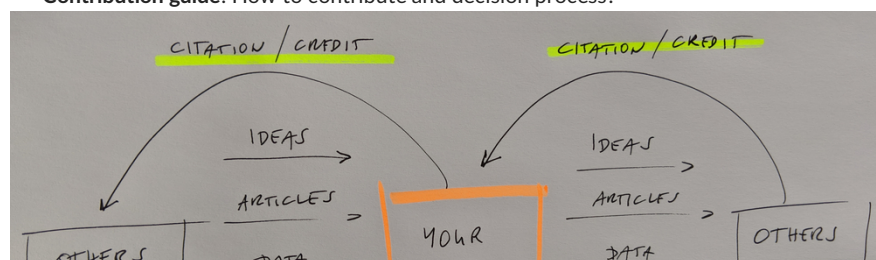
- Have you experienced an implicit expectation of support?
- Supporting all requests can lead to overworking and mental health issues.
- Not supporting requests can also induce guilt.
- Most projects are maintained by 1 or 2 persons.

Instructor note Most projects connected to Question 4 (above) can be viewed as "casual" contributors. Interests change. "Casual contributors are like tourists visiting NYC for a weekend" (Nadia Asparouhova, book below).

- **Date of last code change:** Is the project abandoned?
- If you maintain all projects that you start forever, at some point it may be difficult to start new projects.
- **Release history:** How about stability and backwards-compatibility?
- **Versioning:** Will it be painful to upgrade?
- What are your experiences? Do you agree with the above thoughts?
- **Number of open pull requests and issues:** Are they followed-up?
- Book recommendation: Nadia Asparouhova (formerly Nadia Eghbal): "Working in Public: The Making and Maintenance of Open Source Software (Stripe Press)"
- **Example:** Will it be difficult to get started?

How our work connects to the work of others

- **Contribution guide:** How to contribute and decision process?

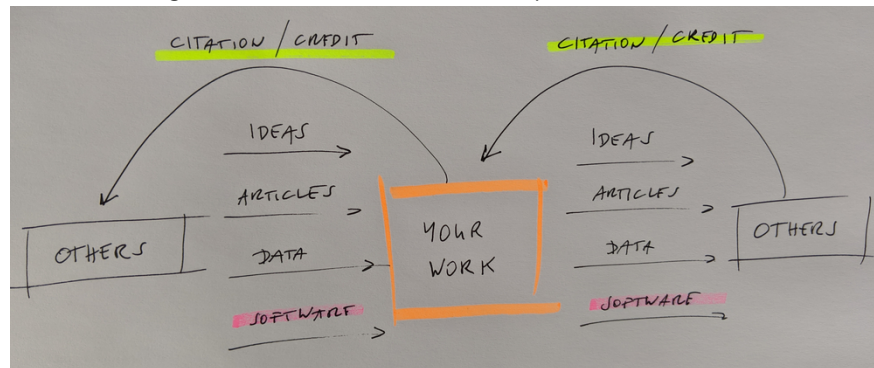


able to reuse your stuff. When you find a repository with code you would like to reuse, you may look at the following things to determine its reusability:

- Have you experienced an implicit expectation of support?
 - Supporting all requests can lead to overworking and mental health issues.
 - Not supporting requests can also induce guilt.
 - Most projects are maintained by 1 or 2 persons.
- This is not a project, it's a community. Contributors for a longer time. Interests change. "Casual contributors are like tourists visiting NYC for a weekend" (Nadia Asparouhova, book below).
- **Date of last code change:** Is the project abandoned?
 - If you maintain all projects that you start forever, at some point it may be difficult to start new projects.
 - **Release history:** How about stability and backwards-compatibility?
 - **Versioning:** Will it be painful to upgrade?
 - What are your experiences? Do you agree with the above thoughts?
 - **Number of open pull requests and issues:** Are they followed-up?
 - Book recommendation: Nadia Asparouhova (formerly Nadia Eghbal): "Working in Public: The Making and Maintenance of Open Source Software (Stripe Press)"
 - **Installation instructions:** Will it be difficult to get it running?
 - **Example:** Will it be difficult to get started?

How our work connects to the work of others

- **Contribution guide:** How to contribute and decision process?



Whether and what we can share depends on how we obtained the components.

- Our work depends on outputs from others. Research of others depends on our outputs.
- Whether you can share your output depends on how you obtained your input.
- A repository that is private today might become public one day.
- Sometimes "OTHERS" are you yourself in the future in a different group/job.
- Our code often starts by changing other code: **derivative work**.
- **Software licenses** matter. And this is what we will discuss in the next episode.

Software licensing focusing on open source

Objectives

- Principles of open source licensing
- Difference between permissive and copyleft licenses
- Regulations for AI-generated and AI-assisted code
- Determine the software license for your project following EU regulation
- Navigate the Joinup Licensing Assistant to select a compliant license
- Understand the licensing distinction between container recipes and container images

Research Software Alliance Policy Directory

This lesson is designed as practical educational material for researchers and research software engineers.

Introduction: What is a Software License?

Under copyright law, worldwide software without an explicit license automatically defaults to All Rights Reserved, meaning nobody else has the legal right to run, modify, embed, or cite your code. A software license is a legal permission, grant, agreement, and/or contract that overrides this statutory default, defining how downstream researchers can reuse your work.

This lesson covers only the general principles of open-source reuse, copyright scope, and how we adapt to open-source licenses to define both how we grant permissions for software we develop (outbound licensing) and how we safely comply with terms attached to code written by others (inbound reuse).

You need to make informed references below and legal experts, especially if you have legal services at your host institute, could be of help:

- Open-source licenses fall into two main families:
 - **Permissive** (e.g., **MIT**, **Apache-2.0**, **BSD**): *Do whatever you want, just keep credit.* Grants maximum reuse freedom, allowing anyone to modify, embed, or re-license your code in open or closed projects.
 - **Copyleft/Reciprocal** (e.g., **GPL-3.0**, **EUPL-1.2**): *Share alike.* Grants full freedom to run and modify, but mandates that any distributed derivative or combined work must also be released under matching copyleft terms. Often informally referred to as **viral** or **infectious** because its open-source requirements propagate across code boundaries (such as embedding snippets or static linking) into downstream projects. The diagram

This lesson is designed as practical educational material for researchers and research

Introduction: What is a Software License?

[illegible]

- This lesson covers only the general principles of open-source reuse, copyright scope, and software adaptation.
- In this lesson, we focus on open-source licenses to define both how we grant permissions for software we develop (outbound licensing) and how we safely comply with terms attached to code written by others (inbound reuse).
- If you need formal guidance, references below and legal experts, especially if you have legal services at your host institute, could be of help:
- Open-source licenses fall into two main families:
 - **Permissive (e.g., MIT, Apache-2.0, BSD):** *Do whatever you want, just keep credit.*
 - [EU Directive 2009/24/EC](#)
 - Grants maximum reuse freedom, allowing anyone to modify, embed, or re-license your code in open or closed projects.
 - **Copyleft/Reciprocal (e.g., GPL-3.0, EUPL-1.2):** *Share alike.* Grants full freedom to run and modify, but mandates that any distributed derivative or combined work must also be released under matching copyleft terms. Often informally referred to as *viral* or *infectious* because its open-source requirements propagate across code boundaries (such as embedding snippets or static linking) into downstream projects. The diagram below unifies these license choices and their downstream rights:

```
%%{init: {'themeVariables': { 'edgeLabelBackground': '#faf5ff' }}}%%
```

flowchart TB

```

subgraph box["How License Selection Governs Code Reuse"]
  A["<b>Your Research Codebase </b></i>(Source code, container recipes, prompt templates)</i>"] -->|"No Licen

  A -->|"Attach License</>(Explicit Permission Grant)"| C["Select License"]

  C -->|"Goal: Maximum adoption & unrestricted reuse"| D["<b>Permissive License</b>"]
  C -->|"Goal: Ensure changes stay open-source (Reciprocity)"| E["<b>Copyleft License</b>"]
  C -->|"Goal: Proprietary control & restricted access"| F["<b>Closed Source / Restricted</b></>❌ <i>Flavour n

  D --> D1["Run & Modify? <b>Yes!</b>"]
  D --> D2["Embed in closed product? <b>Yes!</b>"]
  D --> D3["Must changes stay open? <b>No</b> (Optional)"]

  E --> E1["Run & Modify? <b>Yes!</b>"]
  E --> E2["Embed in closed product? <b>No!</b>"]
  E --> E3["Must changes stay open? <b>Yes!</b> (Mandatory)"]
end

classDef green fill:#e6ffe6,stroke:#2b8a3e,stroke-width:2px,color:#1b4332;
classDef red fill:#ffe3e3,stroke:#e03131,stroke-width:2px,color:#5c0000;
classDef yellow fill:#fff9db,stroke:#f59f00,stroke-width:2px,color:#5c3c00;
classDef white fill:#f8f9fa,stroke:#adb

Protected Code Expression: The source code, variable names, APIs,
classDef blue fill:#e6f2ff,stroke:#007bff,stroke-width:2px,color:#0056b3;
classDef purple fill:#f8d7da,stroke:#dc3545,stroke-width:2px,color:#8b0000;
classDef defaultState fill:#ffe3e3,stroke:#e03131,stroke-width:2px,color:#5c0000;
classDef box fill:#ffffff,stroke:#adb5bd,stroke-width:1px;

```

```
class D1,D2,D3,E1,E3 green;
class D4,D5,D6,D7,D8,D9,D10,D11,D12,D13,D14,D15,D16,D17,D18,D19,D20,D21,D22,D23,D24,D25,D26,D27,D28,D29,D30,D31,D32,D33,D34,D35,D36,D37,D38,D39,D40,D41,D42,D43,D44,D45,D46,D47,D48,D49,D50,D51,D52,D53,D54,D55,D56,D57,D58,D59,D60,D61,D62,D63,D64,D65,D66,D67,D68,D69,D70,D71,D72,D73,D74,D75,D76,D77,D78,D79,D80,D81,D82,D83,D84,D85,D86,D87,D88,D89,D90,D91,D92,D93,D94,D95,D96,D97,D98,D99,D100,D101,D102,D103,D104,D105,D106,D107,D108,D109,D110,D111,D112,D113,D114,D115,D116,D117,D118,D119,D120,D121,D122,D123,D124,D125,D126,D127,D128,D129,D130,D131,D132,D133,D134,D135,D136,D137,D138,D139,D140,D141,D142,D143,D144,D145,D146,D147,D148,D149,D150,D151,D152,D153,D154,D155,D156,D157,D158,D159,D160,D161,D162,D163,D164,D165,D166,D167,D168,D169,D170,D171,D172,D173,D174,D175,D176,D177,D178,D179,D180,D181,D182,D183,D184,D185,D186,D187,D188,D189,D190,D191,D192,D193,D194,D195,D196,D197,D198,D199,D200,D201,D202,D203,D204,D205,D206,D207,D208,D209,D210,D211,D212,D213,D214,D215,D216,D217,D218,D219,D220,D221,D222,D223,D224,D225,D226,D227,D228,D229,D230,D231,D232,D233,D234,D235,D236,D237,D238,D239,D240,D241,D242,D243,D244,D245,D246,D247,D248,D249,D250,D251,D252,D253,D254,D255,D256,D257,D258,D259,D260,D261,D262,D263,D264,D265,D266,D267,D268,D269,D270,D271,D272,D273,D274,D275,D276,D277,D278,D279,D280,D281,D282,D283,D284,D285,D286,D287,D288,D289,D290,D291,D292,D293,D294,D295,D296,D297,D298,D299,D300,D301,D302,D303,D304,D305,D306,D307,D308,D309,D310,D311,D312,D313,D314,D315,D316,D317,D318,D319,D320,D321,D322,D323,D324,D325,D326,D327,D328,D329,D330,D331,D332,D333,D334,D335,D336,D337,D338,D339,D340,D341,D342,D343,D344,D345,D346,D347,D348,D349,D350,D351,D352,D353,D354,D355,D356,D357,D358,D359,D360,D361,D362,D363,D364,D365,D366,D367,D368,D369,D370,D371,D372,D373,D374,D375,D376,D377,D378,D379,D380,D381,D382,D383,D384,D385,D386,D387,D388,D389,D390,D391,D392,D393,D394,D395,D396,D397,D398,D399,D400,D401,D402,D403,D404,D405,D406,D407,D408,D409,D410,D411,D412,D413,D414,D415,D416,D417,D418,D419,D420,D421,D422,D423,D424,D425,D426,D427,D428,D429,D430,D431,D432,D433,D434,D435,D436,D437,D438,D439,D440,D441,D442,D443,D444,D445,D446,D447,D448,D449,D450,D451,D452,D453,D454,D455,D456,D457,D458,D459,D460,D461,D462,D463,D464,D465,D466,D467,D468,D469,D470,D471,D472,D473,D474,D475,D476,D477,D478,D479,D480,D481,D482,D483,D484,D485,D486,D487,D488,D489,D490,D491,D492,D493,D494,D495,D496,D497,D498,D499,D500,D501,D502,D503,D504,D505,D506,D507,D508,D509,D510,D511,D512,D513,D514,D515,D516,D517,D518,D519,D520,D521,D522,D523,D524,D525,D526,D527,D528,D529,D530,D531,D532,D533,D534,D535,D536,D537,D538,D539,D540,D541,D542,D543,D544,D545,D546,D547,D548,D549,D550,D551,D552,D553,D554,D555,D556,D557,D558,D559,D560,D561,D562,D563,D564,D565,D566,D567,D568,D569,D570,D571,D572,D573,D574,D575,D576,D577,D578,D579,D580,D581,D582,D583,D584,D585,D586,D587,D588,D589,D590,D591,D592,D593,D594,D595,D596,D597,D598,D599,D600,D601,D602,D603,D604,D605,D606,D607,D608,D609,D610,D611,D612,D613,D614,D615,D616,D617,D618,D619,D620,D621,D622,D623,D624,D625,D626,D627,D628,D629,D630,D631,D632,D633,D634,D635,D636,D637,D638,D639,D640,D641,D642,D643,D644,D645,D646,D647,D648,D649,D650,D651,D652,D653,D654,D655,D656,D657,D658,D659,D660,D661,D662,D663,D664,D665,D666,D667,D668,D669,D670,D671,D672,D673,D674,D675,D676,D677,D678,D679,D680,D681,D682,D683,D684,D685,D686,D687,D688,D689,D690,D691,D692,D693,D694,D695,D696,D697,D698,D699,D700,D701,D702,D703,D704,D705,D706,D707,D708,D709,D710,D711,D712,D713,D714,D715,D716,D717,D718,D719,D720,D721,D722,D723,D724,D725,D726,D727,D728,D729,D730,D731,D732,D733,D734,D735,D736,D737,D738,D739,D740,D741,D742,D743,D744,D745,D746,D747,D748,D749,D750,D751,D752,D753,D754,D755,D756,D757,D758,D759,D760,D761,D762,D763,D764,D765,D766,D767,D768,D769,D770,D771,D772,D773,D774,D775,D776,D777,D778,D779,D780,D781,D782,D783,D784,D785,D786,D787,D788,D789,D790,D791,D792,D793,D794,D795,D796,D797,D798,D799,D800,D801,D802,D803,D804,D805,D806,D807,D808,D809,D810,D811,D812,D813,D814,D815,D816,D817,D818,D819,D820,D821,D822,D823,D824,D825,D826,D827,D828,D829,D830,D831,D832,D833,D834,D835,D836,D837,D838,D839,D840,D841,D842,D843,D844,D845,D846,D847,D848,D849,D850,D851,D852,D853,D854,D855,D856,D857,D858,D859,D860,D861,D862,D863,D864,D865,D866,D867,D868,D869,D870,D871,D872,D873,D874,D875,D876,D877,D878,D879,D880,D881,D882,D883,D884,D885,D886,D887,D888,D889,D890,D891,D892,D893,D894,D895,D896,D897,D898,D899,D900,D901,D902,D903,D904,D905,D906,D907,D908,D909,D910,D911,D912,D913,D914,D915,D916,D917,D918,D919,D920,D921,D922,D923,D924,D925,D926,D927,D928,D929,D930,D931,D932,D933,D934,D935,D936,D937,D938,D939,D940,D941,D942,D943,D944,D945,D946,D947,D948,D949,D950,D951,D952,D953,D954,D955,D956,D957,D958,D959,D960,D961,D962,D963,D964,D965,D966,D967,D968,D969,D970,D971,D972,D973,D974,D975,D976,D977,D978,D979,D980,D981,D982,D983,D984,D985,D986,D987,D988,D989,D990,D991,D992,D993,D994,D995,D996,D997,D998,D999,1000;
class E2 dashed;
class F dashed;
```

Scope of this Lesson: What Counts as *Software*?

class A,C while;
class box box_fill;
Across international legal frameworks (such as 17 U.S.C. § 101 and WIPO-World Intellectual Property Organization model provisions), software is broadly defined as a set of instructions for a computer to perform a task.

Copyright Foundation: Expression vs. Ideas

To understand why licenses are required, you must understand how copyright law treats software. Under EU statutory law (Directive 2009/24/EC) and international treaties, software is protected under copyright as a **literary work**.

- **Source Code:** Original algorithms written from scratch or implemented from scientific papers. However, copyright law draws a sharp, fundamental distinction between what is protected and what is free for anyone to use.
- **Third-Party Integrations:** Embedded permissive or copyleft code snippets and linked libraries (dynamically/statically).
- **Infrastructure as Code:** Ansible playbooks, Terraform configurations, container Recipes (Dockerfile , Appcontainer .def)

- **Protected (Code Expression):** The original source code text, variable names, APIs, class and built-in types, or compiling and for test, code readability, design comments, classDef, default state, fill:#fff3e3,stroke:#e03131,stroke-width:2px,color:#5c0000;
- **Not Protected (Underlying Ideas):** Mathematical algorithms, scientific models, programming logic, data structures, and interface principles.

class D1,D2,D3,E1,E3 green;
Because copyright restricts only the *creative human expression* and not the underlying *ideas* or *algorithms*, developers could use open-source licenses to define the exact terms under which that expression can be legally shared and modified.

Scope of this Lesson: What Counts as Software?

class F dashed;
class B dashed red;
class A,C white;
class box box_fill;
Across international legal frameworks (such as 17 U.S.C. § 101 and WIPO-World Intellectual Property Organization model provisions), software is broadly defined as a set of instructions to be used directly or indirectly on a computer to bring about a certain result.

To understand why licenses are required, you must understand how copyright law treats software. Because modern research software extends beyond simple Python scripts, this lesson applies copyright and licensing principles across six core research software assets:

- **Source Code:** Original algorithms written from scratch or implemented from scientific papers. However, copyright law draws a sharp, fundamental distinction between what is protected and what is free for anyone to use.
- **Third-Party Integrations:** Embedded permissive or copyleft code snippets and linked libraries (dynamically/statically).
- **Infrastructure as Code:** Ansible playbooks,Terraform configurations,container Recipes (`Dockerfile` , Apptainer `.def`).
- **Container Images:** Bundled binary filesystem snapshots (`.sif` files, OCI registry images).
- **AI-Assisted Code:** Code generated, refactored, or assembled with human creative oversight.
- **AI Prompt Templates:** Complex, engineered system prompts and structured frameworks meeting the threshold of human creative authorship.

Motivation: Debugging a License Compliance Failure

With the understanding of the difference between Permissive and Copyleft licenses, examine what happens when they collide inside an automated CI/CD pipeline:

```
%%{init: {'themeVariables': { 'edgeLabelBackground': '#faf5ff' }}}%%
flowchart TB
    subgraph box["CI/CD License Compliance Debugging Pipeline"]
        A["Paste snippet copied from somewhere"] --> A2["<b>Build Trigger:</b>Push to my-code-base"]
        A2 --> B["Run Compliance Scanner"]
        B --> C["Check Inbound vs.</b>Outbound Terms"]
        C --> D["<b>BUILD FAILURE</b><br>Pasted c"]
        D --> E["Select Patch Option"]
        E --> F["<b>BUILD FAIL</b><br>Comments do not"]
        E --> G["<b>BUILD PASS</b><br>Your license matches"]
        E --> H["<b>BUILD PASS</b><br>New code expr"]
        E --> I["<b>PASSED SCANNER (LEGAL TRAP!)</b><br>In"]
    end
```

Limitations of AI-Assisted Licensing Advice

Modern software is permissive (e.g., MIT, Apache 2.0, BSD) or copyleft (e.g., GPL, AGPL). Claude, GitHub Copilot, to generate boilerplate, refactor functions, and answer project setup questions.

```
P -->|"I want to use"| C
However, using these tools for legal compliance introduces a subtle risk of AI legal bias as AI models are overwhelmingly trained on US-centric web data and legal forum posts, their output default almost universally to US common law concepts such as Fair Use, Work Made for Hire, and Derivative Works.
classDef pass fill:#e0ffe0,stroke:#2b8a3e,stroke-width:2px,color:#1b4332;
classDef copyleft fill:#fff9db,stroke:#f59f00,stroke-width:2px,color:#5c3c00;
classDef fail fill:#ffe3e3,stroke:#e03131,stroke-width:2px,color:#5c0000;
classDef warning fill:#fff3bf,stroke:#f08c00,stroke-width:2px,color:#5c0000;
class D,F fail;
class P,G,H pass;
class CL copyleft;
class I warning;
```

Standardizing In-File Declarations: SPDX Identifiers

Selecting a license is only half the battle; automated scanners and CI/CD pipelines need a machine-readable way to verify license compliance per file without parsing long legal texts.

Limitations of AI-Assisted Licensing Advice

Modern software developers routinely rely on AI assistants (e.g., GitHub Copilot, Claude, Gemini) to generate boilerplate, refactor functions, and answer project setup questions. While these tools are powerful, they often lack the nuanced understanding of legal contexts and jurisdictional requirements that a human lawyer would provide. This reliance introduces a subtle risk of **AI legal bias** as AI models are overwhelmingly trained on US-centric web data and legal forum posts, their outputs defaulting almost universally to **US common law concepts** such as *Fair Use*, *Work Made for Hire*, and *Derivative Works*. In contrast, the European Union has a distinct legal framework (e.g., *Software Directive*, 2009/24/EC) with different legal realities related to exceptions, author ownership, and code adaptations. Relying blindly on AI legal advice creates significant compliance blind spots, which is why this lesson equips you with a direct, EU-aligned framework for software licensing.

P -->|"I want to use"| C

Over using the fast tools for legal purposes guidance introduces a subtle risk of **AI legal bias** as AI models are overwhelmingly trained on US-centric web data and legal forum posts, their outputs default almost universally to **US common law concepts** such as *Fair Use*, *Work Made for Hire*, and *Derivative Works*. In contrast, the European Union has a distinct legal framework (e.g., *Software Directive*, 2009/24/EC) with different legal realities related to exceptions, author ownership, and code adaptations. Relying blindly on AI legal advice creates significant compliance blind spots, which is why this lesson equips you with a direct, EU-aligned framework for software licensing.

class P,G,H pass;

class CL copyleft;

class D,F fail;

class I warning;

class A,B,C fail;

class box box_fill;

Standardizing In-File Declarations: **SPDX Identifiers**

Selecting a license is only half the battle; automated scanners and CI/CD pipelines need a machine-readable way to verify license compliance per file without parsing long legal texts.

Managed by the Linux Foundation, **SPDX identifiers** (Software Package Data Exchange) provide standardized short tags (e.g., `MIT` , `Apache-2.0` , `GPL-3.0-only` , `EUPL-1.2`) placed at the very top line of every source file:

```
# SPDX-License-Identifier: MIT
# Copyright (c) 2026 Author Name <author@institute.eu>
```

Throughout the exercise scenarios below, look for the **In-File Identification (SPDX)** callouts to see how these tags apply directly to Python scripts, container recipes, and engineered prompt templates.

License Selection Decision Matrix & Scenario Index

To help you navigate open-source compliance, the matrix below serves as an upfront quick-reference summary and interactive index for the core licensing scenarios encountered in research software engineering.

Our decision framework is grounded in the European Commission's **Joinup Licensing Assistant (JLA)**, which evaluates software assets across six criteria categories: **Can** (Permissions), **Must** (Obligations), **Cannot** (Restrictions), **Compatible** (Interoperability), **Law** (Jurisdiction), and **Support** (Governance).

Module 1: Clean Slate: Authoring Original Code & Algorithms

When writing original code or implementing published mathematical logic, you control 100% of your copyright. This module focuses on the **Authoring Original Code & Algorithms** scenario, which allows you to jump directly to its detailed exercise, legal analysis, and JLA selection instructions.

Scenario Module	Demonstrated Path / Focus	Compliant Target Licenses
Scenario 1: Authoring original code and algorithms	Permissive Focus (Author Choice)	<code>MIT</code> , <code>Apache-2.0</code> , <code>BSD-3-Clause</code>
You wrote an original algorithm from scratch (in Python, C++, Rust, etc.). Your repository contains only your original source code and dependency specifications (<code>requirements.txt</code> , <code>CMakeLists.txt</code> , <code>Cargo.toml</code>). 4. Embed Copyleft Mandatory Copyleft <code>EUPL-1.2</code> , <code>GPL-3.0</code> 5. Licensing Goal: You want maximum adoption and zero friction for commercial or academic reuse. Legal Reality: External dependencies remain separate works. Because you have not bundled third-party code inside your repository, you hold full copyright over your original codebase. 6. Container Recipe Permissive Focus <code>MIT</code> , <code>Apache-2.0</code> , <code>BSD-3-Clause</code> 7. JLA Selection Strategy: Multi-License Bundle Governed by individual licenses, your original authorship while granting them maximum flexibility to incorporate your code into their own proprietary software, you require citation credit (Incl. Copyright) without imposing share-alike conditions (leaving <code>Copyleft/Share a.</code> unselected). 9. Prompt Chaining Architecture Permissive Focus <code>MIT</code> , <code>Apache-2.0</code> ✓ Solution		

Module 1: Clean Slate - Authoring Original Code & Algorithms [Return to any module link](#)
to jump directly to its detailed exercise, legal analysis, and JLA selection instructions.
When writing original code or implementing published mathematical logic, you control 100%

of your copyright.

Scenario Module

Demonstrated Path / Focus

Compliant Target Licenses

 Scenario 1: Authoring original code and algorithms

MIT

Apache-2.0

BSD-3-Clause

You wrote an original algorithm from scratch (in Python, C++, Rust, etc.). Your repository contains only your original source code and dependency specifications (`requirements.txt` , `CMakeLists.txt` , `Cargo.toml`).

4. **Embed Copyleft**  Mandatory Copyleft

- **Licensing Goal:** You want **maximum adoption** and zero friction for commercial or academic reuse.

5. **License Choice**  Mandatory Copyleft

- **Legal Reality:** External dependencies remain separate works. Because you have not bundled third-party code inside your repository, you hold full copyright over your original codebase.

7. **JLA Selection Strategy:**  Multi-license Bundle

8. **Both Open and Proprietary Software**  Permissive Focus (Author Choice)

9. **Prompt Chaining Architecture**  Permissive Focus

✓ Solution

What to select in the JLA interface:

1. **Can Column:** Select `Distribute` , `Modify/merge` , and `Commercial use`

2. **Must Column:** Select `Incl. Copyright`

3. **Support Column:** Select `OSI approved`

- **JLA Filter Matches:** `MIT` , `Apache-2.0` , `BSD-3-Clause`
- **Permissive vs. Public Domain (EU Civil Law Nuance):** Public domain dedications (e.g., `CC0` , `Unlicense`) attempt to give away all rights. However, under EU civil law, authors cannot legally give up their moral rights (*droit moral*). Selecting an explicit permissive license like `MIT` or `Apache-2.0` grants broad permissions globally, remains legally valid under European copyright law, and guarantees academic citation credit.
- **Downstream Obligations:** Anyone who reuses, modifies, or integrates your code into their work must preserve your copyright notice and license text. They are not required to share their modifications or open-source their downstream projects.
- **Allowed Inbound Snippets:** If you want to include small third-party code snippets in your files, you can freely embed code licensed under **permissive terms** (e.g., MIT, BSD, Apache-2.0, 0BSD) or public domain waivers (CC0) without affecting your permissive license. However, embedding copyleft snippets (e.g., GPL, EUPL) will trigger reciprocal obligations, forcing your entire repository to be re-licensed under those copyleft terms.

• **In-File Identification (SPDX):** Apply standard machine-readable SPDX identifier to implement attribution and enforce reciprocity under matching reciprocal terms.

• **JLA Selection Strategy:** To enforce reciprocal sharing, you must mandate that downstream distributors disclose their modified source code (`Disclose source`) and

```
# SPDX-License-Identifier: MIT
# Copyright (c) 2026 Author Name <author@institute.eu>

import numpy as np
```

What to select in the JLA interface:

1. **Can Column:** Select `Distribute` , `Modify/merge` , and `Commercial use`

 2. **Must Column:** Select `Incl. Copyright` , `Disclose source` , and `Copyleft/Share a.`

3. **Support Column:** Select `OSI approved`

You developed a custom mathematical solver implementing algorithms from academic literature. You want to ensure that any downstream improvements, extensions, or modifications made by others are shared back with the scientific community.

- **JLA Filter Matches:** `EUPL-1.2` , `GPL-3.0` , `AGPL-3.0`
- **Copyleft Mechanics (EUPL vs. GPL Nuance):** `GPL-3.0` is the standard global copyleft license, but `EUPL-1.2` is specifically tailored for European institutions.
- **Life Cycle Considerations:** If you plan to publish in a peer-reviewed journal, preprints, or provide built-in compatibility with GPL, and explicitly default to sharing within state jurisdiction and courts.
- **Legal Reality & Obligations:** Anyone who distributes themselves a modified version, but you must provide implementations to help preserve copyright. Applying the copyleft type legally binds and preserve your digital copyright versions of your

- **In-File Identification (SPDX):** Apply standard machine-readable SPDX identifier implementation to release their source code under matching reciprocal terms.
- **License Selection Strategy:** At the top of your scripts, you must mandate that downstream distributors disclose their modified source code (`Disclose source`) and

```
# SPDX-License-Identifier: MIT
# Copyright (c) 2026 Author Name <author@institute.eu>
```

```
import numpy as np
```

What to select in the JLA interface:

1. **Can Column:** Select `Distribute` , `Modify/merge` , and `Commercial use`
2. **Must Column:** Select `Incl. Copyright` , `Disclose source` , and `Copyleft/Share a.`
3. **Support Column:** Select `OSI approved`

You developed a custom matrix solver for implementing algorithms from academic literature. You want to ensure that any downstream improvements, extensions, or modifications made by others are shared back with the scientific community.

- **JLA Filter Matches:** `EUPL-1.2` if `GPL-3.0` or `AGPL-3.0` are shared back with the scientific community.
- **Copyleft Mechanics (EUPL vs. GPL Nuance):** `GPL-3.0` is the standard global copyleft license, but `EUPL-1.2` is specifically tailored for European institutions.
- **License Goal:** Officially published in 2008, `EUPL` (share alike) (preventing that parties from third parties, including built-in compatibility proprietary with GPL, and explicitly defaults to sharing the entire code under the same license and courts).
- **Legal Reality & Obligations:** Any code you distribute is yours and is not copyrighted, but you must provide a implementation to fully preserve the copyright. Applying the copyleft type legal and preserve your digital copyrighted versions of your code.
- **Allowed Inbound Snippets:** You can freely embed code snippets licensed under permissive terms (e.g., MIT, Apache-2.0, BSD) or public domain waivers (CC0). You may also embed snippets from compatible copyleft code (e.g., EUPL, GPL). However, you cannot embed closed-source or proprietary code snippets.
- **In-File Identification (SPDX):** Apply standard machine-readable SPDX identifier comments directly at the top of your scripts:

```
# SPDX-License-Identifier: EUPL-1.2
# Copyright (c) 2026 Author Name <author@institute.eu>
```

```
import numpy as np
```

Module 2: The Dependency Minefield – Inbound Code & Linking

Embedding third-party source code snippets or linking against strong copyleft libraries introduces legal boundaries that restrict your repository choices.

🔥 Scenario 3: Embedding permissively licensed third-party code

You are building an RSE tool and copied a helper function or utility snippet from a third-party project licensed under a permissive license (e.g., MIT or Apache-2.0) directly into one of your source files.

- **Notice Preservation Nuance:** Permissive licenses are flexible, but they are not license-free. If you copy code from an Apache-2.0 or BSD-3-Clause project into your MIT-licensed repository, you must retain the original author's copyright.
- **Legal Reality:** Permissive licenses explicitly grant you permission to copy, modify, and embed their code into your repository. However, embedding permissive code does not make the original third-party copyright disappear; you must preserve the original copyright attribution and license terms for that specific snippet.
- **Downstream Obligations:** Downstream users receive your project under your main permissive license (e.g., MIT), but they must preserve both your overall copyright notice and the specific third-party notices attached to embedded snippets.
- **JLA Selection Strategy:** Because inbound permissive code gives you maximum licensing flexibility, your overall repository can remain permissively licensed. To reflect this, select citation obligations (`Incl. Copyright`) without imposing reciprocal sharing constraints (leaving `Copyleft/Share a.` unselected).
- **Allowed Inbound Snippets:** In addition to the embedded permissive snippet, you can freely embed other permissively licensed code (MIT, BSD, Apache-2.0) or public domain waivers (CC0). You cannot embed copyleft code (e.g., GPL, EUPL) without upgrading your entire repository's license to match that copyleft license.
- **In-File Identification (SPDX):** Mark both your overall file license and the specific embedded snippet using SPDX comments:

What to select in the JLA interface:

```
# SPDX-License-Identifier: MIT
# Copyright (c) 2026 Author Name <author@institute.eu>

# --- Embedded Third-Party Snippet ---
# SPDX-License-Identifier: Apache-2.0
# Copyright (c) 2024 External Contributor <dev@external-lib.org>
def fast_matrix_solver(matrix):
    # Embedded algorithm implementation
    return np.linalg.solve(matrix, np.eye(len(matrix)))
# --- End Embedded Snippet ---
```

- **Notice Preservation Nuance:** Permissive licenses are flexible, but they are not license-free. If you copy code from an Apache 2.0 or BSD-3-Clause project into your MIT-licensed repository, you must retain the original author's copyright statement and license identifier directly above the embedded code block.
- **Downstream Obligations:** Downstream users receive your project under your primary permissive license (e.g., MIT), but they must preserve both your overall copyright attribution and license terms for that specific snippet.
- **JLA Selection Strategy:** Because inbound permissive code gives you maximum licensing flexibility, your overall repository can remain permissively licensed. To reflect this, select citation obligations (`Incl`, `Copyright`) without imposing reciprocal sharing constraints (leaving `Copyleft/Share a` unselected).
- **Allowed Inbound Snippets:** In addition to the embedded permissive snippet, you can freely embed other permissively licensed code (MIT, BSD, Apache 2.0) or public domain waivers (CC0). You cannot embed copyleft code (e.g., GPL, EUPL) without upgrading your entire repository's license to match that copyleft license.
- **In-File Identification (SPDX):** Mark both your overall file license and the specific embedded snippet using SPDX comments:

What to select in the JLA interface:

```
# SPDX-License-Identifier: MIT
# Copyright (c) 2026 Author Name <author@institute.eu>

# --- Embedded Third-Party Snippet ---
# SPDX-License-Identifier: Apache-2.0
# Copyright (c) 2024 External Contributor <dev@external-lib.org>
def fast_matrix_solver(matrix):
    # Embedded algorithm implementation
    return np.linalg.solve(matrix, np.eye(len(matrix)))
# --- End Embedded Snippet ---

def main():
    pass
```

🔗 Scenario 4: Embedding copyleft third-party code

You are building an software tool and copied a non-trivial code snippet from a third-party project licensed under a copyleft license (e.g., GPL-3.0 or EUPL-1.2) directly into one of your source files.

- **Licensing Goal:** Comply with legal requirements imposed by the inbound copyleft code while ensuring your overall repository remains legally compliant.
- **Legal Reality:** Copyleft licenses require that any work containing copyleft code must be shared under a compatible copyleft license as a whole. Embedding copyleft code directly into your repository creates a single combined work, making copyleft licensing mandatory for your entire project.
- **JLA Selection Strategy:** Because the inbound copyleft code forces your repository to adopt reciprocal sharing terms, you must configure JLA to require source code disclosure (`Disclose source`) and reciprocal licensing (`Copyleft/Share a.`).

✓ Solution

```
# SPDX-License-Identifier: GPL-3.0-or-later
# Copyright (c) 2026 Author Name <author@institute.eu>

# --- Embedded Copyleft Snippet ---
# SPDX-License-Identifier: GPL-3.0-or-later
# Copyright (c) 2023 External Researcher <researcher@university.org>
def optimized_fft_filter(data_signal):
    # Embedded copyleft algorithm implementation
    return np.fft.fft(data_signal)
# --- End Embedded Snippet ---

def main():
    pass
```

If the embedded snippet is `EUPL-1.2`, its built-in compatibility provisions allow you to license your combined project under `GPL-3.0` if your project ecosystem requires it, resolving license conflicts without violating EUPL terms.

Module 3: Dependency Linking & Packaging

- **Downstream Obligations:** Anyone who receives, modifies, or distributes your repository must receive full access to the source code under the same copyleft license terms (`GPL-3.0` or `EUPL-1.2`) and preserve all copyright notices.
- **Allowed Inbound Snippets:** Because your overall repository is now governed by a copyleft license, you can safely embed code from permissive sources (MIT, BSD, Apache-2.0, CC0) as well as compatible copyleft sources. You cannot embed proprietary, closed-source code or snippets from incompatible copyleft licenses.

- **In-File Identification (SPDX):** Mark your overall file license and clearly cite the embedded copyleft snippet using SPDX comments:

🔗 Scenario 5: Linking against a GPL-licensed library

You are developing an software application that imports or links against an external

```
# SPDX-License-Identifier: GPL-3.0-or-later
# Copyright (c) 2026 Author Name <author@institute.eu>

# --- Embedded Copyleft Snippet ---
# SPDX-License-Identifier: GPL-3.0-or-later
# Copyright (c) 2023 External Researcher <researcher@university.org>
def optimized_fft_filter(data_signal):
    # Embedded copyleft algorithm implementation
    return np.fft.fft(data_signal)
# --- End Embedded Snippet ---

def main():
    pass
```

If the embedded snippet is **EUPL-1.2**, its built-in compatibility provisions allow you to license your combined project under **GPL-3.0** if your project ecosystem requires it, resolving license conflicts without violating EUPL terms.

Module 3: Dependency Linking & Packaging

- Downstream Obligations:** Anyone who receives, modifies, or distributes your repository must receive full access to the source code under the same copyleft license terms (**GPL-3.0** or **EUPL-1.2**) and preserve all copyright notices. When software incorporates external dependencies, whether by dynamic linking, static compiling, or bundling binaries into container images licensing obligations expand beyond your own written source code. This module covers how dependency boundaries, build automation scripts, and packaged container artifacts affect legal compliance under the Joinup Licensing Assistant (JLA) framework.
- Allowed Inbound Snippets:** Because your overall repository is now governed by a copyleft license, you can safely embed code from **permissive sources** (MIT, BSD, Apache-2.0, CC0) as well as **compatible copyleft sources**. You cannot embed proprietary, closed-source code or snippets from incompatible copyleft licenses.

- In-File Identification (SPDX):** Mark your overall file license and clearly cite the embedded copyleft snippet using SPDX comments:

You are developing an software application that imports or links against an external software library licensed under GPL-3.0 (e.g., importing a GPL Python package or linking a C/C++ static/shared library).

- Licensing Goal:** Ensure legal compliance while using copyleft libraries as core dependencies in your software project.
- Legal Reality:** Under mainstream copyright interpretation and the text of GPL-3.0, linking your code directly against a GPL library (whether statically or dynamically) creates a combined work. Consequently, the copyleft obligations of the external library extend to your entire repository.
- JLA Selection Strategy:** Because linking to a GPL library requires your distributed project to be released under matching reciprocal terms, you must configure JLA to mandate source code disclosure (**Disclose source**) and reciprocal licensing (**Copyleft/Share a.**).

✓ Solution

What to select in the JLA interface:

- Can Column:** Select **Distribute**, **Modify/merge**, and **Commercial use**
- Must Column:** Select **Incl. Copyright**, **Disclose source**, and **Copyleft/Share a.**
- Support Column:** Select **OSI approved**

- JLA Filter Matches:** **GPL-3.0**, **EUPL-1.2** environment must satisfy GPL compatibility.
- Linking Boundaries & Strong Selection (Legal Nuance):** Why GPL forces copyleft: Linking against a standard **GPL-3.0** library extends copyleft to your entire project. Your repository must adopt a compatible

```
# SPDX-License-Identifier: GPL-3.0-or-later
# Copyright (c) 2026 Author Name <author@institute.eu>

import gpl_licensed_solver # External GPL dependency forces GPL/EUPL compliance

def solve_system(data):
    return gpl_licensed_solver.compute(data)
```

derivative work), copyleft does not extend to your application. In these dynamic linking scenarios, your own project can stay **permissively licensed** (e.g., MIT, Apache-2.0, BSD). However, note that this EUPL stance reflects Commission

🔧 Scenario 6: Authoring container recipes and environment specifications

- incorporation continues to trigger EUPL copyleft obligations.
- You are creating a **Dockerfile**, Conda **environment.yml**, or build recipe to automate the
- Downstream Obligations:** Downstream users who receive or run your application must receive full access to your source code under **GPL-3.0** (or **EUPL-1.2**), along with all upstream copyright notices and build scripts required to recompile the project.
- Licensing Goal:** You want **maximum adoption** and reuse of your build automation
- Allowed Inbound Code & Dependencies:** Your project can import or include other script so other researchers can freely adapt and build upon your workflow.
- Legal Reality:** Build recipes and configuration scripts are plain-text source code (**CC0**). However, all code linked together in the final executable or runtime separate from the software binaries they download at execution time. You hold

- **JLA Filter Matches:** `GPL-3.0` , `EUPL-1.2` . Environment must satisfy GPL compatibility.
- **Linking Boundaries & Source Selection (Legal Nuance):** Why GPL forces copyleft: Linking against a standard `GPL-3.0` library extends copyleft to your entire project. Your repository must adopt a compatible

```
# SPDX-License-Identifier: GPL-3.0-or-later
# Copyright (c) 2026 Author Name <author@institute.eu>

import gpl_licensed_solver # External GPL dependency forces GPL/EUPL compliance

def solve_system(data):
    return gpl_licensed_solver.compute(data)
```

derivative work), copyleft does not extend to your application. In these dynamic linking scenarios, your own project can stay **permissively licensed** (e.g., MIT, Apache-2.0, BSD). However, note that this EUPL stance reflects Commission

🔑 Scenario 6: Authoring container recipes and environment specifications

- incorporation continues to trigger EUPL copyleft obligations.
- You are creating a `Dockerfile` , `Conda environment.yml` , or build recipe to automate the
- **Downstream Obligations:** Downstream users who receive or run your application setup of your research environment. The recipe itself contains setup instructions, shell commands, and package lists, must receive full access to your source code under `GPL-3.0` (or `EUPL-1.2`), along with all upstream copyright notices and build scripts required to recompile the project.
- **Licensing Goal:** You want **maximum adoption** and reuse of your build automation script so other researchers can freely adapt and build upon your workflow.
- **Allowed Inbound Code & Dependencies:** Your project can import or include other **permissively licensed** packages (MIT, BSD, Apache-2.0) and public domain waivers
- **Legal Reality:** Build recipes and configuration scripts are plain-text source code (CC0). However, all code linked together in the final executable or runtime separate from the software binaries they download at execution time. You hold copyright over the unique build instructions you write in the Dockerfile.
- **JLA Selection Strategy:** To allow anyone to reuse or adapt your container recipe without restrictions, you require citation credit (`Incl. Copyright`) while leaving reciprocal requirements (`Copyleft/Share a.`) unselected.

✓ Solution

What to select in the JLA interface:

1. **Can Column:** Select `Distribute` , `Modify/merge` , and `Commercial use`
2. **Must Column:** Select `Incl. Copyright`
3. **Support Column:** Select `OSI approved`

- **JLA Filter Matches:** `MIT` , `Apache-2.0` , `BSD-3-Clause`
- **Recipe vs. Image Nuance:** The license applied to a `Dockerfile` covers only the recipe instructions, not the software packages installed inside the container when `docker build` runs. A permissively licensed Dockerfile can install both permissive and copyleft packages without legal conflict.
- **Downstream Obligations:** Anyone who reuses or adapts your build recipe must preserve your original copyright notice in the header of the recipe file.
- **Allowed Inbound Snippets:** You can freely include build commands and code snippets from permissively licensed build scripts or public domain code.
- **Legal File Identification (SPDX):** Place SPDX identifier comments at the top of your Dockerfile pre-build file. Embedding pre-installed binaries triggers source-code distribution obligations for any copyleft software (e.g., Linux base packages, coreutils, GPL libraries) pre-installed

```
# SPDX-License-Identifier: MIT
# Copyright (c) 2026 Author Name <author@institute.eu>

FROM ubuntu:24.04
RUN apt-get update && apt-get install -y python3 python3-pip
COPY solver.py /app/solver.py
```

✓ Solution

What to select in the JLA interface:

🔑 Scenario 7: Distributing pre-built container images

1. **Can Column:** Select `Distribute` and `Commercial use`
 2. **Must Column:** Select `Incl. Copyright` and `Disclose source`
 3. **Support Column:** Select `OSI approved`
- ing a compiled Docker image to a public registry (GitHub Container Registry, or an institutional registry) containing an OS layer, runtime binaries, dependencies, and your application code.

- **JLA Filter Matches:** `Multi-License Bundle` (Governed by constituent package terms)
- **Licensing Goal:** Safely distribute compiled container images without violating the license terms of any software layer or binary included inside the image.
- **Multi-License Aggregation Nuance:** Applying a permissive license (like MIT) to your application code inside the container does not override or erase the GPL/LGPL obligations of base system packages installed in `/usr/lib` or `/usr/bin` .

- **Legal File Identification (SPDX):** Place SPDX identifier comments at the top of your Dockerfile or source files. Publishing pre-built binaries triggers source-code distribution obligations for any copyleft software (e.g., Linux base packages, coreutils, GPL libraries) pre-installed

```
# SPDX-License-Identifier: MIT
# Copyright (c) 2026 Author Name <author@institute.eu>

FROM ubuntu:24.04
RUN apt-get update && apt-get install -y python3 python3-pip
COPY solver.py /app/solver.py
```

✓ Solution

What to select in the JLA interface:

🔧 Scenario 7: Distributing pre-built container images

1. **Can Column:** Select **Distribute** and **Commercial use**
 2. **Multi-Column:** Select **Incl. Copyright** and **Disclose source** (ing a compiled Docker image to Docker Hub, GitHub, or an institutional registry) containing
 3. **Support Column:** Select **OSI approved**
- an OS layer, runtime binaries, dependencies, and your application code.

- **JLA Filter Matches:** **Multi-License Bundle** (Governed by constituent package terms)
- **Licensing Goal:** Safely distribute compiled container images without violating the license terms of any software layer or binary included inside the image
- **Multi-License Aggregation Nuance:** Applying a permissive license (like MIT) to your application code inside the container does not override or erase the GPL/LGPL obligations of base system packages installed in **/usr/lib** or **/usr/bin**. Distributing the built image binary makes you a distributor of all installed packages.
- **Downstream Obligations:** You must ensure that downstream users can obtain the source code for copyleft components shipped inside the image, typically by publishing the **Dockerfile** and build steps used to generate the image from public upstream sources.
- **Allowed Inbound Packages:** Before publishing an image binary, run automated compliance scanning tools (e.g., Syft, Trivy) to generate a Software Bill of Materials (SBOM) and verify that no non-redistributable or proprietary software is packaged inside.
- **In-File Identification (Metadata Annotations):** Document the multi-license nature of the aggregate bundle using standard OCI (Open Container Initiative) image labels inside your Dockerfile:

```
# SPDX-License-Identifier: MIT
# Copyright (c) 2026 Author Name <author@institute.eu>

FROM ubuntu:24.04
LABEL org.opencontainers.image.authors="author@institute.eu"
# OCI Standard Image Annotations for Docker Hub Compliance
LABEL org.opencontainers.image.title="My Research Pipeline"
LABEL org.opencontainers.image.licenses="MIT AND GPL-3.0-or-later"
LABEL org.opencontainers.image.vendor="My Institute Name"
LABEL org.opencontainers.image.description="Includes Ubuntu 24.04 base layers (GPL/LGPL) and custom solver (MIT)"

COPY solver.py /app/solver.py
```

generally not eligible for copyright protection under current EU and international legal standards. However, if an LLM reproduces a substantial copyrighted code snippet verbatim from its training data (memorization), that output snippet retains its original copyright and license obligations.

Module 4: Emerging Workflows & AI

- **JLA Selection Strategy:** To ensure maximum adoption and academic reuse for your AI-assisted development tools and machine learning models introduce unique legal challenges regarding copyright ownership, training data provenance, and behavioral constraints. This module **Copyleft/Share a.** license projects supported by AI code generation tools and how to package research software that bundles AI models, weights, and datasets alongside source code.

✓ Solution

What to select in the JLA interface:

🔧 Scenario 8: AI-assisted code generation

1. **Can Column:** Select **Distribute**, **Modify/merge**, and **Commercial use** (s, unit tests, or do)
 2. **Multi-Column:** Select **Incl. Copyright** repository.
 3. **Support Column:** Select **OSI approved**
- **Licensing Goal:** Retain clear ownership and apply a **permissive license** (**MIT** or
 - **JLA Filter Matches:** **MIT**, **Apache-2.0**, **BSD-3-Clause** in copyright infringement or
 - **AI Code Generation & Verification Nuance:** Because training human AI output cannot hold copyright, your copyright applies to the overall project structure, human-written logic, and creative choices. To protect your repository against accidental

```
COPY solver.py /app/solver.py
```

generally not eligible for copyright protection under current EU and international legal standards. However, if an LLM reproduces a substantial copyrighted code snippet

Module 4: Emerging Workflows & AI

AI Selection Strategy: To ensure maximal adoption and academic reuse for your AI-assisted development tools and machine learning models, introduce techniques for your overall codebase, require citation credit (Incl. Copyright), while avoiding share-alike challenges regarding copyright ownership, fair use, and behavioral restrictions. This (headline Copyleft/Share a. license) is supported with a patented compliance package research software that bundles AI models, weights, and datasets alongside source code.

✓ Solution

What to select in the JLA interface:

Scenario 8: AI-assisted code generation

1 **Can Column:** Select: Distribute t, Modify/merge code, Commercial use rs, unit tests, or
2 **Must Column:** Select: Incl. Copyright epository.

3. Support Column: Select **OSI approved**

- **Licensing Goal:** Retain clear ownership and apply a **permissive license** (MIT or Apache-2.0)
- **JNA Filter Matches:** MIT, Apache-2.0, BSD-3-Clause, etc. (no copyright infringement or trademark issues)

Copyright Generation from VerboComba does not create an AI output cannot hold copyright, your copyright applies to the overall project structure, human-written logic, and creative choices. To protect your repository against accidental copyright infringement or copyleft contamination from AI memorization, turn on public code matching filters in your AI tools and run automated code-similarity scanners before releasing your repository.

- **Downstream Obligations:** Downstream users must preserve your copyright notice for the repository. They are free to reuse, modify, and integrate your code into commercial or open-source projects.
- **Allowed Inbound Snippets:** You can include permissively licensed code, public domain code (CC0), and AI-generated snippets that have been verified against verbatim training data duplication.
- **In-File Identification (SPDX):** Apply standard machine-readable SPDX identifier comments directly at the top of your scripts:

```
# SPDX-License-Identifier: MIT
# Copyright (c) 2026 Author Name <author@institute.eu>

def filter_sensor_data(raw_readings: list[float]) -> list[float]:
    """Cleans raw sensor data (written with AI assistance and human review)."""
    return [reading for reading in raw_readings if reading > 0.0]
```

🔧 Scenario 9: Packaging AI workflows, datasets, and model weights

You are likely to find these: MIT, Apache-2.0 (for the source components), and benchmarks (e.g., OpenAI, Salesforce) and benchmark datasets (e.g., HuggingFace, OpenAI, Salesforce).

Code vs. Data/Weights & OpenRAIL Nuance: Never apply software licenses like GPL or MIT to raw datasets or model weights. Use **CC-BY-4.0** or **CC0** for non-

- **Licensing Goal:** Artificially created dual licensing strategies that protect both the software source code and the contributing assets (data, models) are not reasonable and not appropriate for open-source software and cannot be filtered via standard
- **Legal Reality:** Standard open-source software licenses (MIT, GPL) are written around **stream obligations**. Do not legally identify a fork dataset repository for network protection under EU legal framework and give credit for what is not governed by data ownership (e.g. curators, database creation, curation) as is traditional code copyright.
- **JAIS selection:** Strategies use JAIS to elaborate Q&A on how licenses should be licensed with the executable code component **weight models, provided code** and **training data**. **Separate licenses** **from data/weight files** on the dataset and weight files.
- **In-File Identification (SPDX / Dual-Licensing Structure):** Document the dual licensing scheme in your root repository structure and script headers:

```
# SPDX-License-Identifier: MIT
# Copyright (c) 2026 Author Name <author@institute.eu>
#
# Note: Source code is licensed under MIT.
# Model weights in /models/ and datasets in /data/ are licensed under CC-BY-4.0.

import torch

def load_pipeline():
```

- You are a **File Identifier**. MIT ? Apache-2.0 ? For the source code alongside trained machine learning model weights (e.g., **OpenRAIL** **Nuance** ?) and benchmark datasets. Code v1. Data Weights & OpenRAIL ?
- **Licensing Goal:** Apply a dual licensing strategy to **OpenRAIL** to impose software restrictions (e.g., prohibiting specific data uses), while **not** necessarily **do not** apply the GPL to **OpenRAIL** open-source software and cannot be filtered via standard
 - **Legal Reality:** Standard open-source software licenses (MIT, GPL) are written **downstream** obligations. Do **not** legally **use** **source code** for **network** products **under** **EU** **source code** and **source code** **weights** **delivered** by **data** **rights** (e.g., **generative data** **protection**) rather than traditional code copyright.
 - **JLA Selection Strategy:** Use **JLA** to select an **OS** **approved** license **source code** with the **copyright** **license** **components** **weight** **models** **provided** **while** **using** **Creative Commons** **license** **license** **from** **data/weight** **license** **dataset** **and** **weight** **files**.
 - **In-File Identification (SPDX / Dual-Licensing Structure):** Document the dual-licensing scheme in your root repository structure and script headers:

```
# SPDX-License-Identifier: MIT
# Copyright (c) 2026 Author Name <author@institute.eu>
#
# Note: Source code is licensed under MIT.
# Model weights in /models/ and datasets in /data/ are licensed under CC-BY-4.0.

import torch

def load_pipeline():
    model = torch.load("models/climate_weights.safetensors")
    return model
```

Best Practices: Attaching a License to Your Repository

Once you have selected a license using the JLA, you must officially attach it to your repository so automated scanners, package registries, and downstream researchers can verify your terms.

1. Adding the Root LICENSE File

Always place the full text of your chosen license in a plain-text file named `LICENSE` or `LICENSE.txt` at the root of your repository.

- **Exact Legal Text:** Copy the standard text directly from spdx.org/licenses or choosealicense.com.
- **Copyright Header:** Ensure you fill in the copyright year and copyright holder line at the top of the license text:

```
Copyright (c) 2026 [Author Name or Institution Name]
```

1. **Include License Texts:** Place full license files inside a `LICENSES/` directory (e.g., `LICENSES/MIT.txt`; `LICENSES/GPL-3.0-or-later.txt`), of standard licenses (e.g., removing clauses from **CC-BY** or **MIT**). Custom license edits create **non-standard** legal texts that compliance scanners cannot parse, defaulting your repository back to restricted status. **SPDX** tags.
2. **Add in-File SPDX Headers:** Label every source file, build script, and prompt file with **SPDX** tags.
3. **Verify Compliance:** Run the automated REUSE linter in your CI/CD pipeline:

2. Documenting License Status in README.md

```
# Install and run REUSE compliance check
pip install reuse
reuse lint

## License

This project is licensed under the MIT License - see the [LICENSE](LICENSE) file for details.

[![License: MIT](https://img.shields.io/badge/License-MIT-yellow.svg)](https://opensource.org/licenses/MIT)
```

At the end of a project, it is a proactive design choice. By using the **Joinup Licensing Assistant (JLA)** framework to align your repository license with your inbound dependencies from day one, your CI/CD pipeline passes cleanly on the first run.

3. Automated Compliance with the REUSE Standard

For multi-asset research repositories containing code, data, container build recipes, and the diagram below illustrates how selecting a compatible license upfront ensures your code prompt templates follow the **FSFE REUSE Initiative** standard, passes automated compliance checks and results in a legally sound release:

1. **Include License Texts:** Place full license files inside a `LICENSES/` directory (e.g., `LICENSES/MIT.txt`; `LICENSES/GPL-3.0-or-later.txt`), of standard licenses (e.g., removing clauses from GPL or MIT). Custom license edits create non-standard legal texts that compliance scanners cannot parse, defaulting your repository back to restricted status.
2. **Documenting License Status in** `README.md`
3. **Verify Compliance:** Run the automated REUSE linter in your CI/CD pipeline:

```
# Install and run REUSE compliance check
pip install reuse
reuse lint

## License

This project is licensed under the MIT License - see the [LICENSE](LICENSE) file for details.

[![License: MIT](https://img.shields.io/badge/License-MIT-yellow.svg)](https://opensource.org/licenses/MIT)
```

the end of a project, it is a proactive design choice. By using the **Joinup Licensing Assistant (JLA)** framework to align your repository license with your inbound dependencies from day one, your CI/CD pipeline passes cleanly on the first run.

For multi-asset research repositories containing code, data, container build recipes, and The diagram below illustrates how selecting a compatible license upfront ensures your code passes automated compliance checks and results in a legally sound release:

```
%%{init: {'themeVariables': { 'edgeLabelBackground': '#faf5ff' }}}%%
flowchart TB
    subgraph local["1. Local Authoring & Standardization"]
        A["<b>Inbound Reuse Trigger:</b><br>User copies copyleft snippet <i>(Scenario 4)</i><br>or links GPL li"]
        B --> C["<b>Standardize Local Codebase:</b><br>1. Add <b>SPDX Headers</b> to all files <i>(Scenarios 1-9)</i>"]
    end

    subgraph cicd["2. Automated CI/CD & Verification"]
        C --> D["<b>Git Push</b> to Repository"]
        D --> E["<b>Verify Inbound vs.<br>Outbound Terms</b>"]
        E --> F["SPDX Headers & License Match Confirmed!" | F["✔ <b>BUILD PASSES</b><br>Compliance verified automa"]
        F --> SUCCESS["🎉 <b>COMPLIANT OPEN-SOURCE RELEASE</b><br>Legally sound, reproducible & ready for scient"]
    end

    classDef pass fill:#e6ffe6,stroke:#2b8a3e,stroke-width:2px,color:#1b4332;
    classDef neutral fill:#f8f9fa,stroke:#495057,stroke-width:2px,color:#212529;
    classDef box_fill fill:#ffffff,stroke:#adb5bd,stroke-width:1px;

    class F,SUCCESS pass;
    class A,B,C,D,E neutral;
    class local,cicd box_fill;
```

- **Standardized Distribution:** By adding machine-readable **SPDX headers** across every script, Dockerfile, and prompt template, running `reuse lint` in your pipeline confirms 100% legal clarity for the entire scientific community.

Scenario Mapping Across the Pipeline

Software citation (Handling inbound copyleft (Scenario 4 & Scenario 5): When you copy non-trivial copyleft code snippets (e.g., CC BY-SA from Stack Overflow or GPL snippets) or link directly

against a GPL library, your overall project becomes a combined work. Selecting a compatible copyleft license upfront (`GPL-3.0` or `EUPL-1.2`) satisfies the reciprocal sharing terms and allows the pipeline scanner to pass without conflict.

- **Maintaining Permissive Defaults (Scenario 1 & Scenario 3):** If you write original code or embed only permissively licensed snippets (MIT, Apache-2.0, BSD), selecting a permissive license can make your software more accessible to mainstream users maximum adoption freedom while preserving your citation credit.

Is putting software on GitHub/GitLab/... publishing?

Packaging and Build Automation (Scenario 6 & Scenario 7): Keep plain-text build recipes (Dockerfiles) permissively licensed for maximum reuse, while annotating compiled

code-similarity scanners to code, and apply dual-n-code datasets and model



- **Standardized Distribution:** By adding machine-readable **SPDX headers** across every script, Dockerfile, and prompt template, running `reuse lint` in your pipeline confirms 100% legal clarity for the entire scientific community.

Scenario Mapping Across the Pipeline

Software citation

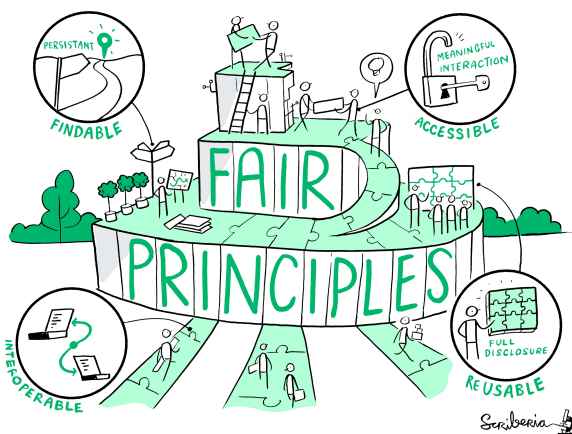
Handling Inborn Copyleft (**Scenario 4 & Scenario 5**): When you copy non-trivial copyleft code snippets (e.g., CC BY-SA from Stack Overflow or GPL snippets) or link directly

against a GPL library, your overall project becomes a combined work. Selecting a compatible copyleft license upfront (`GPL-3.0` or `EUPL-1.2`) satisfies the reciprocal

sharing terms of use on GitHub/GitLab/... publishing?

- **Maintaining Permissive Defaults (**Scenario 1 & Scenario 3**):** If you write original code or embed only permissively licensed snippets (MIT, Apache-2.0, BSD), selecting a permissive license (e.g., `Apache-2.0`) grants downstream users maximum adoption freedom while preserving your citation credit.

Is putting software on GitHub/GitLab/... publishing? Packaging and Build Automation (**Scenario 6 & Scenario 7**): Keep plain-text build recipes (Dockerfiles) permissively licensed for maximum reuse, while annotating compiled container images binaries as multi-licensed separate bundles to satisfy embedded base-



code-similarity scanners to code, and apply dual-n-code datasets and model

FAIR principles. (c) Scriberia for The Turing Way, CC-BY.

Is it enough to make the code public for the code to remain **findable and accessible**?

- No. Because nothing prevents me from deleting my GitHub repository or rewriting the Git history and we have no guarantee that GitHub will still be around in 10 years.
- **Make your code citable and persistent:** Get a persistent identifier (PID) such as DOI in addition to sharing the code publicly, by using services like [Zenodo](#) or similar services.

How to make your software citable

Discussion (Citation-1): Explain how you currently cite software

- Do you cite software that you use? How?
- If I wanted to cite your code/scripts, what would I need to do?

```
cff-version: 1.2.0
message: "If you use this software, please cite it as below."
authors:
  - family-names: Doe
    given-names: Jane
    orcid: https://orcid.org/1234-5678-9101-1121
title: "My Research Software"
version: 2.0.4
doi: 10.5281/zenodo.1234
date-released: 2021-08-11
```

This checklist is adapted from: N. P. Chue Hong, A. Allen, A. Gonzalez-Beltran, et al., Software More about `CITATION.cff` files: Citation Checklist for Developers (Version 0.9.0). Zenodo. 2019b. ([DOI](#))

GitHub now supports CITATION.cff files

Our practical recommendations:

- [Web form to create, edit, and validate CITATION.cff files](#)
- [Xata file called CITATION.cff \(example\)](#)

Papers with focus on scientific software

Get digital object identifier (DOI) for your code on [Zenodo](#) (example).

- [Step-by-step recipe](#) for how to make your GitHub project citable using [Zenodo](#).
- Where can I publish papers which are primarily focused on my scientific software? Great Make it as easy as possible: clearly say what you want cited.

list/summary is provided in this blog post: ["In which journals should I publish my software?"](#)

(Neil P. Chue Hong)

This is an example of a simple `CITATION.cff` file:

How to cite software


```

cff-version: 1.2.0
message: "If you use this software, please cite it as below."
authors:
  - family-names: Doe
    given-names: Jane
    orcid: https://orcid.org/1234-5678-9101-1121
title: "My Research Software"
version: 2.0.4
doi: 10.5281/zenodo.1234
date-released: 2021-08-11

```

This checklist is adapted from: N. P. Chue Hong, A. Allen, A. Gonzalez-Beltran, et al., Software Citation Checklist for Developers (Version 0.9.0). Zenodo. 2019b. (DOI[↗])

Our practical recommendations:

- [Web form to create, edit, and validate CITATION.cff files](#)[↗]
- [Xdata file called CITATION.cff and cffinit](#)[↗]

Papers with focus on scientific software

- [Get a digital object identifier \(DOI\)](#) for your code on [Zenodo](#)[↗] (example[↗]).
- [Step-by-step recipe](#)[↗] for how to make your GitHub project citable using [Zenodo](#)[↗].

Where can I publish papers which are primarily focused on my scientific software? Great

- Make it as easy as possible: clearly say what you want cited.

list/summary is provided in this blog post: ["In which journals should I publish my software?"](#)

([Neil P. Chue Hong](#))[↗]

This is an example of a simple `CITATION.cff` file:

How to cite software

Great resources

- A. M. Smith, D. S. Katz, K. E. Niemeyer, and FORCE11 Software Citation Working Group, "Software citation principles," PeerJ Comput. Sci., vol. 2, no. e86, 2016 (DOI[↗])
- D. S. Katz, N. P. Chue Hong, T. Clark, et al., Recognizing the value of software: a software citation guide [version 2; peer review: 2 approved]. F1000Research 2021, 9:1257 (DOI[↗])
- N. P. Chue Hong, A. Allen, A. Gonzalez-Beltran, et al., Software Citation Checklist for Authors (Version 0.9.0). Zenodo. 2019a. (DOI[↗])
- N. P. Chue Hong, A. Allen, A. Gonzalez-Beltran, et al., Software Citation Checklist for Developers (Version 0.9.0). Zenodo. 2019b. (DOI[↗])

Recommended format for software citation is to ensure the following information is provided as part of the reference (from [Katz, Chue Hong, Clark, 2021](#)[↗] which also contains software citation examples):

- Creator
- Title
- Publication venue
- Date
- Identifier
- Reusable: Software should have clear licensing, documentation, and provenance
- Version (e.g. a standard license and a `README` with usage instructions, authors listed with ORCID(s)).

FAIR for Research Software (FAIR4RS)

There are great resources to self-evaluate the FAIRness of your research software:

The FAIR4RS Principles adapt the original FAIR data principles to improve the **Findability, Accessibility, Interoperability, and Reusability** of research software. The page of how guidelines to software research software more open, discoverable, citable, and reusable, which supports reproducibility and open science. The summary of the FAIR4RS below contains version of the principles listed with version control of: 1. License, 2. Software Principles, 3. Software citation file, 4. Software quality checklist).

Sharing data

- **Findable.** Software should have a globally unique and persistent identifier (e.g. clear versioning of releases, DOI from Zenodo when depositing a software release, etc.).

Services for sharing and collaborating on research data

- **Accessible:** Software and metadata should be retrievable using open protocols (e.g. research data repository like Zenodo), you can search on the [Registry of Research Data Interoperable \(Software\)](#) platform and find format and standards type discipline, other tools
- **Interoperable.** Input/output files in CSV, dependencies listed in `requirements.txt` or `environment.yml`, configuration files in yaml).

- [Zenodo](#)[↗]: A general-purpose open access repository created by OpenAIRE and CERN.

- **Identifiable:** Software should have clear licensing, documentation, and provenance
- **Reusable:** (e.g. a standard license and a `README` with usage instructions, authors listed with ORCIDs).

5. FAIR for Research Software (FAIR4RS)

There are great resources to self-evaluate the *FAIRness* of your research software:

The FAIR4RS Principles adapt the original FAIR data principles to improve the **Findability, Accessibility, Interoperability, and Reusability** of research software. They provide

guidelines to make research software more open, discoverable, citable, and reusable, which supports reproducibility and open science. The summary of the FAIR4RS below is a compact version of the principles listed in [Barber, Micheller et al. "Introducing the FAIR Principles for Software Citation, File, and Software Quality Checklist".](#)

Sharing data

Findable: Software should have a globally unique and persistent identifier (e.g. clear versioning of releases, DOI from Zenodo when depositing a software release).

Services for sharing and collaborating on research data

• **Accessible:** Software and metadata should be retrievable using open protocols (e.g. downloading repository for your data), you can search on the [Registry of Research Data](#).

Interoperable: Software should use file formats and standards to work with other tools

International: Input/output files in CSV, dependencies listed in `requirements.txt` or `environment.yml`, configuration files in yaml).

- **Zenodo**: A general-purpose open access repository created by OpenAIRE and CERN. Integration with GitHub, allows researchers to upload files up to 50 GB.
- **Figshare**: Online digital repository where researchers can preserve and share their research outputs (figures, datasets, images and videos). Users can make all of their research outputs available in a citable, shareable and discoverable manner.
- **EUDAT**: European platform for researchers and practitioners from any research discipline to preserve, find, access, and process data in a trusted environment.
- **Dryad**: A general-purpose home for a wide diversity of datatypes, governed by a nonprofit membership organization. A curated resource that makes the data underlying scientific publications discoverable, freely reusable, and citable.
- **The Open Science Framework**: Gives free accounts for collaboration around files and other research artifacts. Each account can have up to 5 GB of files without any problem, and it remains private until you make it public.

Sweden:

- [Researchdata.se](#) - research data from a wide range of disciplines
- [ICOS for climate data](#)
- [Bolin center climate / geodata](#)
- [NBIS for life science, sequence, omics data](#)

Norway:

Portugal:

- [NIRD archive](#) is widely used by some communities
- [DMP Data Center for Research Data for data](#) kind of data
- [Dataverse.no](#) - Dataverse network, based at University of Tromsø but open for other institutions

Resources for data management

[ELIXIR Norway](#) for life science, sequence, omics data

Denmark

- [Data Stewardship Wizard](#)
- [Datavejviser](#) – metadata catalogue aggregating from research organisations and National Archives

Licensing of datasets and databases

[DTU Data](#) is a national data survey of Denmark and Greenland

- [DTU Data](#) – repository of the Technical University of Denmark
- The EU has a [database directive](#) which restricts data mining on databases.
- [DeC Dataverse for University of Copenhagen](#)
- Has a somewhat similar effect to copyright, because copyright would not apply to data mining.

Finland:

- A good license also gives rights to data mine. So not a major concern.

- [IDA, for general data](#)
- [FSD for social data](#)

- [For information on Finnish resources for open data](#)
- Your country has exceptions for research

Portugal:

- [NIRD archive](#) is widely used by some communities[↗]
- [NDP Portal](#) - [Data Center for Research Data](#) for any kind of data[↗]
- [Dataverse.no](#) - [Dataverse network](#), based at University of Tromsø but open for other institutions[↗]

Resources for data management

[Link to Norway](#) for life science sequence, omics data[↗]

Denmark[↗]

- [Data Stewardship Wizard](#)[↗]
- [Datavejviser](#)[↗] – metadata catalogue aggregating from research organisations and National Archives

Licensing of datasets and databases

[Link to the Danish National Survey of Denmark and Greenland](#)

- [DTU Data](#)[↗] – repository of the Technical University of Denmark
- The EU has a [database directive](#)[↗] which restricts data mining on databases.
- [DataC](#) [Dataverse for University of Copenhagen](#)
- Has a somewhat similar effect to copyright, because copyright would not apply to data mining.

Finland:

- A good license also gives rights to data mine. So not a major concern.

When you can use datasets:

- [IDA, for general data](#)[↗]
- [FSD for social data](#)[↗]
- [More information on Finnish resources for open data](#)[↗]
- The license allows
- Your country has exceptions for research
- The data doesn't come from the EU

License text, slides, images, and supporting information under a [Creative Commons license](#)[↗], and get a DOI using [Zenodo](#)[↗] or [Figshare](#)[↗] or [OSF](#)[↗] other services.

Licensing and machine learning/ AI

This section is maybe more relevant to **developers of AI models / AI systems** rather than **users of AI models / AI systems**.

Is it data? Is it software? It depends. We need to consider the AI system as a whole, the training data, the production data, the AI output, and how it is put on service. **AI models** are like the engine of the car: they cannot do anything without the rest of the car infrastructure. **AI systems** are the whole car with the AI model and all the software and hardware to actually use it.

Depending on what you are going to share, there might be things to consider beyond the license.

For example **large language models** are often shared with open source software licenses, on **HuggingFace** which is like a GitHub/GitLab for AI models (see for example the [OLMO Model](#)[↗]). Many so-called *open-source* models are actually just *open-weights* models: only the trained neural network weights are shared, while the training data, training code, and full documentation are often kept private. This lack of transparency raises concerns about reproducibility and accountability and this phenomenon is sometimes called “**open washing**”

([ref](#)[↗]). Models are also shared with a **model card** which is a documentation tool for transparency that provide a comprehensive snapshot of a model's characteristics and ethical considerations (see [Ch.8 Gilean 2025](#)[↗]).

- [“Expert Q&A on Artificial Intelligence \(AI\) Licensing”](#)[↗]

What about ethics? What about liability?

Further reading

deep network weights) and AI systems (the model with all the software and infrastructure to query it) are becoming more available, there can be legal (and ethical!) requirements on the developer of the AI model/system by the [EU AI Act](#)[↗]. In general

researchers do not need to worry, but ethically one should consider that if the research-purpose AI model/system could be used for something harmful, ethically (if not legally) one should consider if such model/system should be implemented at all.

- The [reproducible research data analysis platform](#)[↗]

Good talks on open reproducible research can be found [here](#)[↗]

What about the training data inside the model? Large models can memorize and unintentionally *“leak”* parts of their training data. This raises concerns about copyright, trade secrets, and personal data. News publishers and artists are suing AI companies for

- [“Top 10 FAIR Data & Software Things”](#)[↗] are brief guides that can be used by the research community to understand how they can make their research (data and software) more

unauthorized use of their content in training. It is still unclear how traditional data licenses can apply to data that has been transformed into model weights. Only the trained neural network weights are shared, while the training data, training code, and full documentation are often kept private. This lack of transparency raises concerns about reproducibility and accountability and this phenomenon is sometimes called “**open washing**”

([ref](#)). Models are also shared with a **model card** which is a documentation tool for transparency that provide a comprehensive snapshot of a model's characteristics and ethical considerations (see [Chris Clerman 2025](#)).

- [“Expert Q&A on Artificial Intelligence \(AI\) Licensing”](#)

What about ethics? What about liability?

Further reading

As deep network weights) and AI systems (the model with all the software and infrastructure to query it) are becoming more available, there can be legal (and ethical!) requirements on the developer of the AI model/system by the [EU AI Act](#). In general researchers do not need to worry, but ethically one should consider that if the research-purpose AI model/system could be used for something harmful, ethically (if not legally) one should consider if such model/system should be implemented at all.

- [The reproducible research data analysis platform](#)

Good talks on open reproducible research can be found [here](#).
What about the training data inside the model? Large models can memorize and unintentionally favour parts of their training data. This raises concerns about copyright, trade secrets, and personal data. News publishers and artists are suing AI companies for

- [“Top 10 FAIR Data & Software Things”](#) are brief guides that can be used by the research community to understand how they can make their research (data and software) more FAIR.
- [Five recommendations for fair software](#)
- [Publishing research software](#) A MIT libraries webpage on why to publish software, where to publish software, and how to make software citable.
- [Software Quality Checklist](#)
- [MolSSI Best Practice Guides](#)
- [Five recommendations for fair software](#)
- [Awesome Research Software Registries](#)

Who is the course for?

- Someone writing their own relatively small material
- Someone who wants to incorporate other code/libraries into their own projects
- Group leader who wants to decide how to balance openness and long-term strategy

List of exercises

Full list

This is a list of all exercises and solutions in this lesson, mainly as a reference for helpers and instructors. This list is automatically generated from all of the other pages in the lesson. Any single teaching event will probably cover only a subset of these, depending on their interests.

Hints

Instructor guide

Before you begin, it's worth thinking about some particular good or bad cases you know about from your experience.

Basics

- When have you lost access or not been able to use something because licensing wasn't handled well.
- When did multiple ownership mess up your plans?
There are the following main acts:

This talk used to be called “software licensing”. You could mention that to make it more interesting, we have adopted the current broader name, and our emphasis is not just a discussion that licensing alone isn't enough. If you want people to use your work, you need to do more to make your work easily reusable.

Software licensing

Emphasis on the concept of derivative work and that everything is based on other things. And if your goal is citations, you want as many people using your work as possible.

Objectives

- Overview of license types. There are many online resources, and the biggest question is knowing about what derivative work is and whether we can share it.
- Get familiar with terminology around licensing. The [cake analogy for licenses](#) is only linked and can be skipped and does not have to be discussed in detail.
- Practical advice for software licensing.
- Then a brief section on code citations. It's good if you manage to start a discussion around this topic.

Copyright

The “sharing data” part is meant to bring awareness about FAIRness and Open Science and to show how DOIs can be obtained for Git repositories.

Hints

Instructor guide

Before you begin, it's worth thinking about some particular good or bad cases you know about from your experience.

Basics

- When have you lost access or not been able to use something because licensing wasn't handled well.
- When did multiple ownership mess up your plans?
There are the following main acts:

This talk used to be called "software licensing". You could mention that to make it more interesting, we have adopted the current broader name, and our emphasis is not just a discussion that licensing alone isn't enough. If you want people to use your work, you need to do more to make your work easily reusable.

Software licensing

Emphasis of the concept of derivative work and that everything is based on other things.

And if your goal is citations, you want as many people using your work as possible,

Objectives

- Overview of license types. There are many online resources, and the biggest question is knowing about what derivative work is and whether we can share it.
- Get familiar with common license around reuse. The [cake analogy for licenses](#) is only linked and can be skipped and does not have to be discussed in detail.
- Practical advice for software licensing.
- Then a brief section on code citations. It's good if you manage to start a discussion around this topic.

Copyright

The "sharing data" part is meant to bring awareness about FAIRness and Open Science and to show how DOIs can be obtained for Git repositories.



- Trademark:** Protects a name/brand from impersonation.
- Patent:** Protects a novel, non-obvious, technical invention.
- Copyright:** Protects **creative expression**: software, writing, graphics, photos, certain datasets, this presentation. Practically "forever" (lifetime of author + 70 years).

Copyright controls whether and how we can distribute the original work or the **derivative work**.

Derivative work: Sampling/remixing



[Midjourney, CC-BY-NC 4.0]

- Changing and distributing software is similar to changing and distributing music
- You can do almost anything if you don't distribute it

Often we don't have the choice:

- We are expected to publish software



[Midjourney, CC-BY-NC 4.0]

- Changing and distributing software is similar to changing and distributing music
- You can do almost anything if you don't distribute it

Often we don't have the choice:

- We are expected to publish software
- Sharing can be good insurance against being locked out

Exercise: Derivative work

💬 Licensing-1: What constitutes derivative work?

This question 5 below can be used as a starting point and copied to the collaborative document or form input for an online poll:

```
## Question 5: Which of these are derivative works?

**Choose many**. Vote by adding an `o` character:

- A. Download some code from a website and add on to it
  - votes:

- B. Download some code and use one of the functions in your code
  - votes:

- C. Changing code you got from somewhere
  - votes:

- D. Extending code you got from somewhere
  - votes:

- E. Completely rewriting code you got from somewhere
  - votes:

- F. Rewriting code to a different programming language
  - votes:

- G. Linking to libraries (static or dynamic), plug-ins, and drivers
  - votes:

- H. Clean room design (somebody explains you the code but you have never seen it)
  - votes:

- I. You read a paper, understand algorithm, write own code
  - votes:
```

Copyright (computer programs are) and/or intellectual properties like trademarks, and inventions (which may include specific technical implementations of systems or code) protected by patents.

✓ Solution

In research ethics, plagiarism is one of the three definition of research misconduct (along with fabrication and falsification, see ALLEA, [European Code of Conduct for Research Integrity](#)). Plagiarism is not illegal per se, but it can lead to serious consequences like the retraction of published work. One can engage in plagiarism, without necessarily breaking any IPR law (e.g. write a new book by reusing the plot of an old book that is not under copyright anymore). Copyright infringement instead is illegal and it can result in criminal charges (e.g. fines). Copyright however protects the particular expression of an idea or fact (for example, the particular source code of a program, but not the underlying algorithm itself).

There is no pre-defined “number of lines of code”, “seconds of a song”, or “pixels of an

- F. Rewriting code to a different programming language
 - votes:
- G. Linking to libraries (static or dynamic), plug-ins, and drivers
 - votes:
- H. Clean room design (somebody explains you the code but you have never seen it)
 - votes:
- I. You read a paper, understand algorithm, write own code
 - votes:

Copyright (computer programs are not, and by default, processes and algorithms), and inventions (which may include specific technical implementations of systems or code) protected by patents.

✓ Solution

In research ethics, plagiarism is one of the three definition of research misconduct (along

- Derivative work: A-F
- Not derivative work: G-I

Integrity (✓). Plagiarism is not illegal per se, but it can lead to serious consequences like the

retraction of published work. One can engage in plagiarism, without necessarily breaking any IPR law (e.g. write a new book by reusing the plot of an old book that is not under copyright anymore). Copyright infringement instead is illegal and it can result in criminal charges (e.g. fines). Copyright however protects the particular expression of an idea or **this** (see example, the code left as reading of a program, but not the underlying algorithm itself).

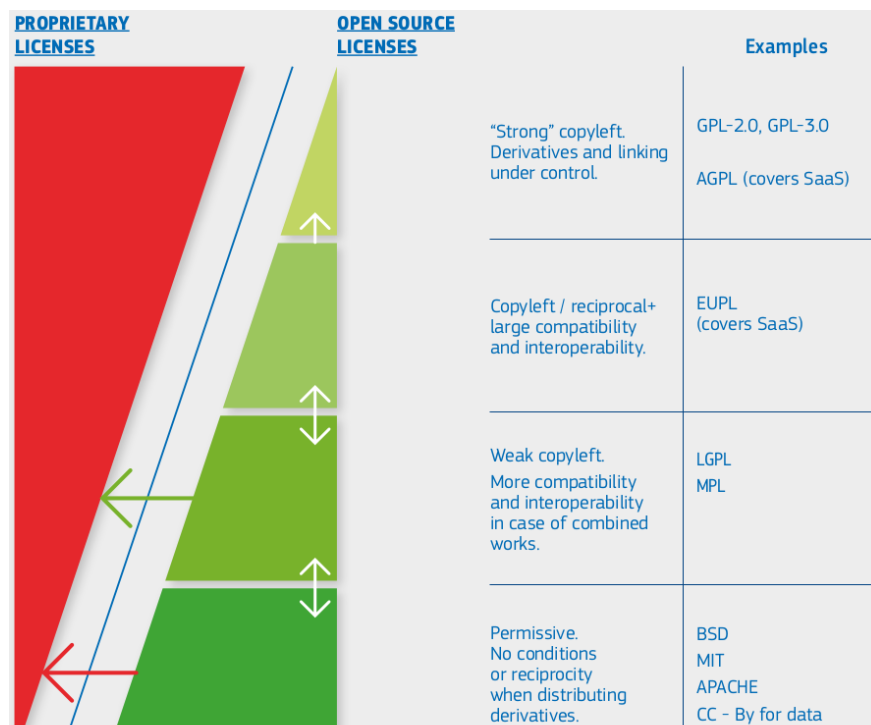
There is no pre-defined “number of lines of code”, “seconds of a song”, or “pixels of an image” that can clearly set the basis for plagiarism or IPR infringement. However in the context of research, it can be possible to use *Quotation Exception* (in EU, [ref](#)) and *Fair use* (in USA, [ref](#)). Fair use has become controversial recently as it is used as legal basis for training large language models based on scraped internet data (See for example Henderson, P., Li, X., Jurafsky, D., Hashimoto, T., Lemley, M. A., & Liang, P. (2023). Foundation models and fair use. *Journal of Machine Learning Research*, 24(400), 1-79. [ref](#))

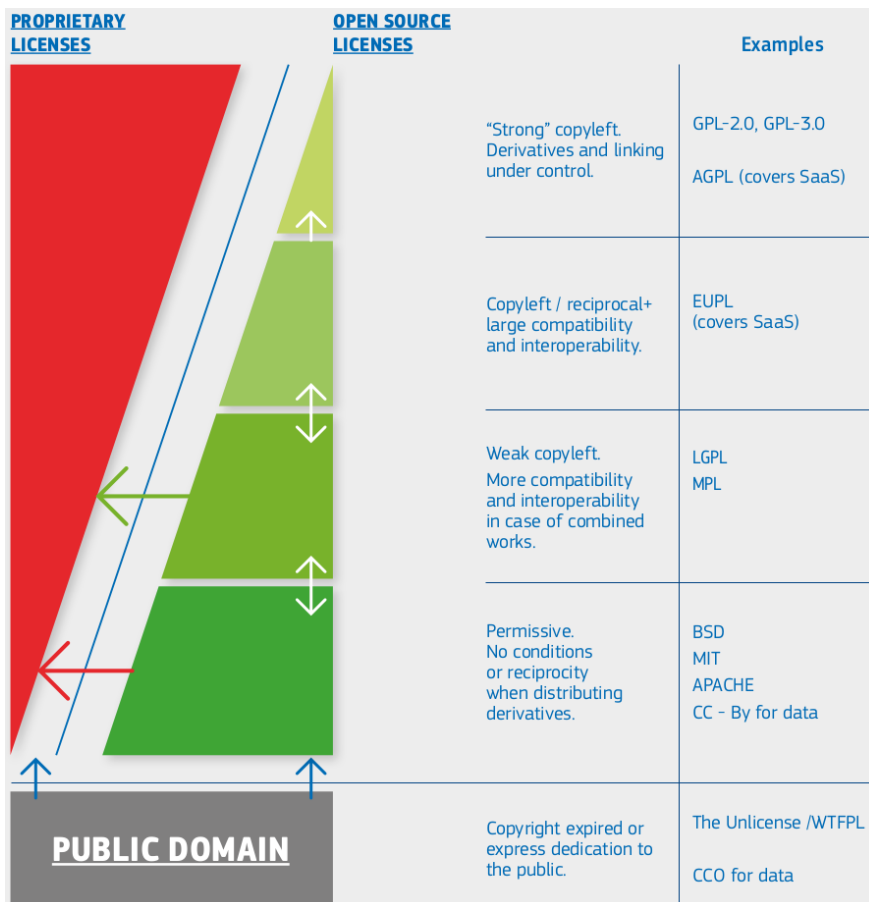
Derivative work and containers

Containers are a bit more tricky when it comes to licenses.

- Distribution of container recipes: it's like distributing source code
- Distribution of container images: it can be considered like distributing a binary compiled software

The latter case is a bit more nuanced and the interested reader should read more about “Mere Aggregation” at [GPL-FAQ](#). Briefly, if the container image just bundles separate programs that talk through normal system interfaces, it is an **aggregate** and each keeps its own license (like a CD-ROM with various packages). If the components are tightly integrated into one program (e.g. a pipeline with various parts that the container can run as a single program), the image may be treated as a **derivative work**, and stricter license obligations (e.g. GPL copyleft) can apply.





European Commission, Directorate-General for Informatics, Schmitz, P., European Union Public Licence (EUPL): guidelines July 2021, Publications Office, 2021,

<https://data.europa.eu/doi/10.2799/77160>

Comments:

- Arrows represent compatibility (A -> B: B can reuse A)
- Proprietary/custom: Derivative work typically not possible (no arrow goes from proprietary to open)
- Permissive: Derivative work does not have to be shared
- Copyleft/reciprocal: Derivative work must be made available under the same license terms
- NC (non-commercial) and ND (non-derivative) exist for data licenses but not really for software licenses

Great resource for comparing software licenses: [Joinup Licensing Assistant](#)

- Do licenses apply if you don't distribute your code? Why or why not?
- Provides comments on licenses
- Easy to compare licenses (example)

[Joinup Licensing Assistant - Compatibility Checker](#)

✓ Solution

- Not biased by some company agenda

1. As indicated [here](#), all publicly accessible user contributions are licensed under Creative Commons Attribution-ShareAlike license. See Stackoverflow [Terms of service](#) for more detailed information.

Exercise: Licensing situations

- Standard licensing rules apply. So in this case, you would need to remove the portion of code you have copied from another package before being able to release your software.
- Consider some common licensing situations
- When the source code is authorized to use the content of a repository when there is no license. And derivative work is also not possible by default. Other risks it may not be clear whether you can reuse and distribute (publish) the buffered code. For a portion of the code from another package, when you have forgotten. Care for the code owners it may not be clear whether they can use and distributed the buffered code. However, the authors may have forgotten to add a license. You want to take a bug report from someone else has released, but there is no license. What's the best way to contact the authors (e.g. make an issue) and ask whether they are willing to add a license?
- How to add a license
- As mentioned in 3, the easiest is to file an issue and explain the reasons why you would like to use this software (or update it).
- Combining software with different licenses can be tricky and it is important to understand compatibilities (or lack of compatibilities) of the various licenses. GPL

- 7. Do licenses really apply if you don't distribute your code? Why or why not?
- 8. Which licenses are most/least attractive for companies with proprietary software?

Easy to compare licenses ([example](#))

[Joinup Licensing Assistant - Compatibility Checker](#)

- ✓ **Solution**
- Not biased by some company agenda

1. As indicated [here](#), all publicly accessible user contributions are licensed under [Creative Commons Attribution-ShareAlike](#) license. See [Stackoverflow Terms of service](#) for more detailed information.

Exercise: Licensing situations

- "Standard" licensing rules apply. So in this case, you would need to remove the portion of code you have copied from another package before being able to release your software.

1. **Exercise:** Consider some common licensing situations

- What if the source repository license to use the content of a repository when there is no license. And derivative work is also not possible by default. Other risks it may have a portion of the code from another package, which you have forgotten. Can you satisfy the journal's request?
- You want to use a blog where the authors may have forgotten to add a license or worse, where they are there.
- How would you ask someone to add a license?
- As mentioned in FS, GPL and BSD is to fill an issue and explain the reasons why some licenses can you pick for your project.
- Combining software with different licenses can be tricky and it is important to understand the compatibilities (or lack of compatibilities) of the various licenses. GPL license is the most protective (BSD and MIT are quite permissive) so for the resulting combined software you could use a GPL license. However, re-licensing may not be necessary.
- Derivative work would need to be shared under this strong copyleft license (e.g. AGPL or GPL), unless the components are only plugins or libraries.
- If you keep your code for yourself, you may think you do not need a license. However, remember that in most companies/universities, your employer is "owning" your work and when you leave you may not be allowed to "distribute your code to your future self". So the best is always to add a license!
- The least attractive licenses for companies with proprietary software are licenses where you would need to keep an open license when creating derivative work. For instance GPL and AGPL. The most attractive licenses are permissive licenses where they can reuse, modify and relicense with no conditions. For instance MIT, BSD and Apache License.

When should I add a license?

Choose a license early in the project, even before you publish it. Later in the project it may become complicated to change it. Agreeing on a software license does not mean that you have to make it open immediately. You can also follow the **"open core" approach**: You don't have to open source all your work. Core can be open and on a public branch. Unpublished code can be on a private repository.

- Create a `LICENSES/` folder in your project and "put the unmodified license text (i.e., the license text template without any copyright notices) in your `LICENSES/` folder. However, we recommend to work as if the code is public, even if it may be private (thanks to E. Glerean for this great suggestion): This is to avoid surprises about code in the <https://reuse.software/faq/#license-templates>). This way if you reuse code from history with incompatible license years later when you decide to open the project, multiple projects, you can keep there multiple license files.

- Put the code that you incorporate into a separate file or separate files. This makes it later easier to see what was incorporated, and what was written from scratch. On top of

How to add a license if your work is derivative work

the file(s) which you have incorporated into your project add (and adapt) the following header (more examples): Your code is derivative work if you have started from an existing code and made changes to it or if you incorporated an existing code into your code.

```
# SPDX-FileCopyrightText: 2023 Jane Doe <jane@example.com>
#
# SPDX-License-Identifier: MIT
```

these two resources:

- The [REUSE](#) initiative was started by the [Free Software Foundation Europe](#) to make licensing of software projects easier. It is OK if you prefer to not follow this strict format
- [Joinup Licensing Assistant - Compatibility Checker](#) but the advantage of following it is that the [reuse-tool](#) makes it then easy to verify and

update license headers if you have many files from different sources.

If the original code does not have a license, you may not be able to distribute your derivative code. You can try to contact the authors and ask them to clarify the license of their code.

If it does not make sense to have several files in your project (e.g. when incorporating something into a notebook), then add a note/comment about the license and where the code came from on top of the functions **small into your own project** with a license that allows you to do so (as an example incorporating a function or two from another project):

- Although it is not dictated by the license but it can still be nice to acknowledge the

• Create a `LICENSES/` folder in your project and “put the unmodified license text (i.e., the license text template without any copyright notices) in your `LICENSES/`”. It may be private (thanks to E. Gleean for this great suggestion): This is to avoid surprises about code in the history with incompatible license years later when you decide to open the project. <https://reuse.software/faq/#license-templates>). This way if you reuse code from multiple projects, you can keep there multiple license files.

• Put the code that you incorporate into a separate file or separate files. This makes it later easier to see what was incorporated, and what was written from scratch. On top of the file(s) which you have incorporated into your project add (and adapt) the following header (more examples): Your code is derivative work if you have started from an existing code and made changes to it or if you incorporated an existing code into your code.

```
# SPDX-FileCopyrightText: 2023 Jane Doe <jane@example.com>
#
# SPDX-License-Identifier: MIT
```

these two resources:

- The [REUSE](#) initiative was started by the [Free Software Foundation Europe](#) to make licensing of software projects easier. It is OK if you prefer to not follow this strict format but the advantage of following it is that the [reuse-tool](#) makes it then easy to verify and update license headers if you have many files from different sources.
- [Joinup Licensing Assistant - Find and compare software licenses](#)
- [Joinup Licensing Assistant - Compatibility Checker](#)

If the original code does not have a license, you may not be able to distribute your derivative code. You can try to contact the authors and ask them to clarify the license of their code.

- If it does not make sense to have several files in your project (e.g. when incorporating something into a notebook), then add a note/comment about the license and where the code came from on top of the functions **small into your own project** with a license that allows you to do so (as an example incorporating a function or two from another project): Although it is not dictated by the license but it can still be nice to acknowledge the incorporated functions/code in your README/documentation and to cite their work if you publish a paper about your code.
- Some licenses are more permissive (you can keep your changes private) but some licenses require you to publish the changes (share-alike).

Practical steps for making **changes to an existing project** with a license that allows you to do so:

- If the project is on GitHub or GitLab or similar, first fork the project (copy it into your user space where you can make changes).
- For the BSD and MIT licenses you are not obliged to state your changes but it can still be helpful for others if you do. You can state your changes in the header of the files you have modified. It can be helpful to state bigger-picture changes in the README file of the project.
- Some licenses are more permissive (you can keep your changes private) but some licenses require you to publish the changes (share-alike).

If your work is not derivative work

If you have started “from scratch”, and not used any existing code, or incorporated existing code into your code, then you may consider your code to be not derivative work.

Before you may choose a license, clarify the following points with, for example, your supervisor, collaborators, or principal investigator:

- Add copyright and license information to each file following <https://reuse.software/tutorial/> which uses a standard format with so-called **SPDX**
- Does your work contract, grant or collaboration agreement dictate a specific license?
- Is there an intent to commercialize the code?

```
# SPDX-FileCopyrightText: 2023 Jane Doe <jane@example.com>
#
# SPDX-License-Identifier: EUPL-1.2
```

Do not invent your own license. Choose one of the standard licenses, otherwise

compatibility is not clear:

- The [REUSE](#) initiative was started by the [Free Software Foundation Europe](#) to make licensing of software projects easier. It is OK if you prefer to not follow this strict format but the advantage of following it is that the [reuse-tool](#) makes it then easy to verify and update license headers if you have many files from different sources.
- [Joinup Licensing Assistant - Find and compare software licenses](#)
- [Joinup Licensing Assistant - Compatibility Checker](#)

For really small projects with one or two files the above may seem excessive and some projects choose to not have copyright information on top of their files and they only have one `LICENSE` file and that is OK for really small projects.

- Create `LICENSES/` folder (example).
- Put the unmodified license text (i.e., the license text template without any copyright notices) in plain text format into the folder (example).
- **ⓘ Licensing code produced by generative AI systems** Here are the two above licenses

With generative AI tools for coding (but the GitHub Copilot, Copyright notices which that implementation to ChatGPT, Claude, Grok, ...) the responsibility fully lays on the person who is going to use (and publish) the generated code. You can never blame the autopilot

Add copyright and license information to each file following supervisor, collaborators, or principal investigator.

<https://reuse.software/tutorial/> which uses a standard format with so-called **SPDX**

- Does your work contract, grant, or collaboration agreement dictate a specific license?
- Is there an intent to commercialize the code?

```
# SPDX-FileCopyrightText: 2023 Jane Doe <jane@example.com>
#
# SPDX-License-Identifier: EUPL-1.2
```

Do not invent your own license. Choose one of the standard licenses, otherwise

compatibility is not clear:

- The **REUSE** initiative was started by the **Free Software Foundation Europe** to make licensing of software projects easier. It is OK if you prefer to not follow this strict format
- [Joinup Licensing Assistant - Find and compare software licenses](#) but the advantage of following it is that the [reuse-tool](#) makes it then easy to verify and
- [Joinup Licensing Assistant - Compatibility Checker](#) update license headers if you have many files from different sources.

For really small projects with one or two files the above may seem excessive and some projects choose to not have copyright information on top of their files and they only have

- one **LICENSE** file and that is OK for really small projects.

- Create a **LICENSES/** folder (example).
- Put the unmodified license text (i.e., the license text template without any copyright

! Licensing code produced by generative AI systems notices) in plain text format into the folder (). Here are the two above licenses

With generative AI tools for coding such as GitHub Copilot, Copilot, or other basic chat implementations (GPT, Claude, Grok, ...) the responsibility fully lays on the person who is going to use (and publish) the generated code. You can never blame the autopilot or the company who invented it, only the driver (you!).

There are various risks in using generative AI code (this is not a taxonomy). A few examples:

- Risks for the derivative work: you think your code is doing what you asked, but you did not review it and your results are false
- Risks for the system in use: your generated code has software security issues, e.g. an import is a *typosquat* of an actual library (e.g. "microsoft" is spelled "rnicrosoft" and depending on the font you might totally miss it...)
- Risks related to licenses/IPR: you have generated code that is actually verbatim copy of fully copyrighted code, or code that requires a strict copyleft license. Plagiarism (ethics) also applies.

If we focus on the last one, a recent paper ([ref](#)) estimates that around 2% of AI generated code is "strikingly similar to existing open-source implementations". Generative AI tools are typically not able to provide an exact reference of where certain bits of generated code were copied from, so it is the responsibility of the researcher to verify that the produced code is citing and referencing the license of other published pieces of software. Possibly, future AI systems for code generation can be trained on code that share the same set of licenses (e.g. based only on MIT) to mitigate these risks.

Great resources

- [Contributor License Agreements](#)
- [4OSS recommendations](#)
- [Research institution policies to support research software \(compiled by the Research Software Association\)](#)
- [Intellectual Property Rights \(IPR\), Licensing And Patents](#)
- [Guidelines on the Aalto University in Finland: Options to Use Software at Aalto University](#)
- [https://researchsoftware.nl/](#) (can send automatic pull request to your GitHub repo)
- <https://biojuringassistant.io/Find-and-choose-software-licenses2.html>
- <http://licensingassistant.org/Compatibility/Check-reference.html>
- [Nadia Asparuhova \(formerly Nadia Eggball\): "Working in Public: The Making and Social Coding Lesson material" by CodeKarmery](#)
- [Maintenance of Open Source Software" \(Stripe Press\)](#)
- [Citation File Format \(CFF\)](#)
- [Open Source Guides](#)
- [License Selector](#)
- [The Architecture of Open Source Applications](#)
- [GitHub licensing guide](#)
- [Christopher M. Kelly: "Two Bits: The Cultural Significance of Free Software" \(Duke University Press, 2008\)](#)
- [Understanding Open Source and Free Software Licensing](#)
- [Open Source \(Almost\) Everything](#)
- [Software Licenses in Plain English](#)
- [99 ways to ruin an open source project](#)
- [Don's Bibliography of Ethical Source Reading and Resources](#)
- [Open Source Casebook](#)
- [Mikko Välimäki: The Rise of Open Source Licensing](#)

! Keypoints

- [Aalto IPR Cheatsheet](#)

Contributor License Agreements

Great resources

- 4OSS recommendations
- Research institution policies to support research software (compiled by the Research Software Alliance)
- Software Property Rights (IPR), Licensing And Patents
- Guidelines for the Aalto University Finland: Option to license software at Aalto University
- Drafts/Research software can send automatic pull request to your GitHub repo)
- https://bioengineeringeducation.org/education/curriculum-topics/2.html
- https://licensingAssistant.com/compatibility/Checker.html
- Nadia Asanogluova (formerly Nadia Eghbal): "Working in Public: The Making and Maintenance of Open Source Software" (Stripe Press)
- Citation File Format (CFF)
- Open Source Guides
- License Selector
- The Architecture of Open Source Applications
- GitHub licensing guide
- Christopher M. Kelty: "Two Bits: The Cultural Significance of Free Software" (Duke University Press, 2008)
- Understanding Open Source and Free Software Licensing
- Open Source (Almost) Everything
- Software Licenses in Plain English
- 99 ways to ruin an open source project
- Don's Bibliography of Ethical Source Reading and Resources
- Open Source Casebook
- Mikko Välimäki: The Rise of Open Source Licensing

Keypoints

- Aalto IPR Cheatsheet
- You cannot ignore licensing: default is "no one can make copies or derivative works".

Reusing

This lesson material is free to reuse, adapt, and cite. This page collects everything you need to do so.

How to cite this lesson

If you use this lesson material, please cite it using this DOI:

https://doi.org/10.5281/zenodo.16410766

The machine-readable citation record is kept in CITATION.cff at the root of this lesson's repository, which lists the full set of contributing authors and stays up to date as the lesson evolves.

Structured metadata

This lesson also publishes its metadata as machine-readable Bioschemas / schema.org structured data, embedded as JSON-LD on every page of this site.

All of this lesson's metadata is generated from a single source, metadata.yml, rendered here directly at build time:

Learning resource type	lesson
License	Social coding and open software - What can you do to get credit for your code and allow reuse?
Website template	CodeRefinery, Radovan Bast, Kjartan Thor Wikfeldt, Johan Hellsvik, Richard Darst, Sabry Razick, Jyry Suvilehto, Anne Fouilloux, Oxana Smirnova, Lucy Whalley, Enrico Glelean
Authors	The website template is maintained by CodeRefinery and rendered with sphinx-lesson: structured lessons with sphinx.
Version	2026-09-10
DOI	10.5281/zenodo.16410766
Instructional material	
License	CC-BY-4.0
Lesson website	All CodeRefinery instructional material is made available under the Creative Commons Attribution license (CC-BY-4.0). The following is a human-readable summary of (and not a substitute for) the full legal text of the CC-BY-4.0 license: https://coderefinery.github.io/social-coding
Source repository	https://github.com/coderefinery/social-coding
Keywords	software licensing, software citation
You are free:	
Educational level	Beginner
Share	to share - copy and redistribute the material in any medium or format
Adapt	to adapt - remix, transform, and build upon the material
Language	en-UK
Is part of	https://coderefinery.org

for any purpose, even commercially. The licensor cannot revoke these freedoms as long as you follow these license terms:

Learning resource type	lesson
License	Social coding and open software - What can you do to get credit for your code and allow reuse?
Website template	CodeRefinery, Radovan Bast, Kjartan Thor Wikfeldt, Johan Hellsvik, Richard Darst, Sabry Razick, Jyry Suvilehto, Anne Fouilloux, Oxana Smirnova, Lucy Whalley, Enrico Glierean
Authors	The website template is maintained by CodeRefinery and rendered with sphinx-lesson : structured lessons with sphinx .
Version	2024-08-10
DOI	10.5281/zenodo.16410766
Instructional material	
License	CC-BY-4.0
Lesson Website	All CodeRefinery instructional material is made available under the Creative Commons Attribution license (CC-BY-4.0) . The following is a human-readable summary of (and not a substitute for) the full legal text of the CC-BY-4.0 license .
Source Repository	https://github.com/coderefinery/social-coding
Keywords	software licensing, software citation
You are free:	
Educational Level	Beginner
Share	to copy and redistribute the material in any medium or format
Adapt	to remix, transform, and build upon the material
Language	en, uk
Is part of	https://coderefinery.org

for any purpose, even commercially. The licensor cannot revoke these freedoms as long as you follow these license terms:

- **Attribution** - You must give appropriate credit (mentioning that your work is derived from work that is Copyright (c) CodeRefinery and, where practical, linking to <https://coderefinery.org> , provide a [link to the license](#) , and indicate if changes were made. You may do so in any reasonable manner, but not in any way that suggests the licensor endorses you or your use.

No additional restrictions - You may not apply legal terms or technological measures that legally restrict others from doing anything the license permits. With the understanding that:

- You do not have to comply with the license for elements of the material in the public domain or where your use is permitted by an applicable exception or limitation.
- No warranties are given. The license may not give you all of the permissions necessary for your intended use. For example, other rights such as publicity, privacy, or moral rights may limit how you use the material.